GeoCrop: Automated Crop Monitoring and Forecasting Toolkit

Project Team

Huzaifa Bin Khalid 22I-2332

Taha Bashir Awan 23I-0824

Saneed Ullah 23I-2568

Session 2023-2027

Supervised by

Mr. Muhammad Aadil Ur Rehman

Co-Supervised by

Mr. Obaid Ur Rehman

Institute of Space Technology (External)



FAST School of Computing

National University of Computer and Emerging Sciences
Islamabad, Pakistan

October, 2026

Project ID: F26-155-GeoCrop

Contents

Students' Submission i

Anti-Plagiarism Statement iii

Authors' Declaration v

Abstract vii

Acknowledgments ix

Executive Summary xi

List of Abbreviations xiii

1 Introduction and Project Vision 1

1.1 Project Overview . . . 1
1.2 Problem Statement . . . 1
1.3 Problem Context and Motivation . . . 2
1.4 Project Vision . . . 2
1.5 Project Objectives . . . 2
1.6 Project Scope . . . 3
1.7 Project Modules . . . 4
1.8 Stakeholders and User Classes . . . 5
1.9 Success Criteria . . . 5
1.10 Constraints and Assumptions . . . 6
1.11 Ethical, Legal and DEI Considerations . . . 6

2 Research and Existing Solutions 7

2.1 Research Method / Approach . . . 7
2.2 Existing Solutions / Products . . . 7
2.3 Comparison of Existing Solutions . . . 9
2.4 Limitations and Gaps . . . 9
2.5 Proposed Solution . . . 10
2.6 Complex Computing Problem . . . 11

2.7 Technical Challenges . . . 11
2.8 Research Findings and Design Decisions . . . 12

3 Proposal Feedback and Iteration Plan **13**
3.1 Proposal Feedback Summary . . . 13
3.2 Proposal Action Register . . . 14
3.3 Iteration Strategy . . . 14
3.4 Iteration 1 Plan . . . 14
3.5 Iteration 2 Plan . . . 16
3.6 Work Breakdown and Responsibilities . . . 18
3.7 Progress Against Plan . . . 18
3.8 Delays, Deviations and Recovery Plan . . . 18

4 Software Requirements **19**
4.1 User Interactions / Use Cases . . . 19
4.2 Use Case Diagram . . . 20
4.3 Detailed Use Cases . . . 20
4.4 Functional Requirements . . . 20
4.4.1 Module 1: Processing Pipeline (Ingestion and Jobs) . . . 20
4.4.2 Module 2: Model Registry and Crop Mapping . . . 22
4.4.3 Module 3: Catalogue, Storage and Statistics . . . 22
4.4.4 Module 4: Web GIS Portal . . . 23
4.5 Non-Functional Requirements . . . 23
4.5.1 Reliability . . . 23
4.5.2 Usability . . . 23
4.5.3 Performance . . . 24
4.5.4 Security . . . 24
4.6 Acceptance Criteria . . . 24
4.7 Requirements Traceability . . . 24

5 System Design **27**
5.1 System Architecture . . . 27
5.2 Architectural Components . . . 27
5.3 Interfaces and Dependencies . . . 27
5.4 Architecture Decisions and Rationale . . . 27
5.5 Design Evolution . . . 27
5.6 Domain Model . . . 31
5.7 Data Design . . . 32
5.8 Process / Behavior Design . . . 32

6 Iteration 1: Design and Implementation **35**
6.1 Iteration Objectives . . . 35
6.2 Selected Use Cases / Features . . . 35

6.3 Design . . . 35
6.4 Implementation . . . 36
6.5 Completed Features / Module . . . 37
6.6 Technical Implementation Details . . . 37
6.7 Algorithms and Technical Decisions . . . 37
6.7.1 Platform algorithms . . . 37
6.7.2 Crop-type evaluation protocol . . . 39
6.7.3 Results: three runs on AOI1 . . . 39
6.7.4 Run 3: improvement ladder . . . 40
6.7.5 Run 3 results in both areas . . . 41
6.7.6 Comparison with published work . . . 41
6.8 Tools, Frameworks and Technologies . . . 43
6.9 Demonstration Evidence . . . 43
6.9.1 Workflow feasibility and resource requirements . . . 46
6.10 Iteration Outcome . . . 47

7 Testing and Early Validation **49**
7.1 Testing Strategy . . . 49
7.2 Test Levels and Techniques . . . 49
7.3 Test Cases . . . 49
7.4 Test Results . . . 49
7.5 Acceptance Testing / Validation . . . 49
7.6 Edge and Failure Cases . . . 52
7.7 Defects and Corrective Actions . . . 53
7.8 Current Validation Status . . . 53

8 Project Management and Development Evidence **55**
8.1 Work Completed . . . 55
8.2 Work Remaining . . . 55
8.3 Individual Responsibilities . . . 56
8.4 Contribution of Each Team Member . . . 56
8.5 Repository and Version Control Evidence . . . 57
8.6 Milestones and Commit Evidence . . . 57
8.7 Tools and Tool-Assisted Development . . . 57
8.8 External Components and Dependencies . . . 58
8.9 Risks and Mitigation . . . 58
8.10 Security and Privacy Considerations . . . 58
8.11 Current Limitations . . . 60

9 Remaining FYP Plan **61**
9.1 Remaining Scope . . . 61
9.2 Remaining Modules and Features . . . 61
9.3 Remaining Iterations . . . 62

9.4 Implementation Plan 62
9.5 Testing and Validation Plan 62
9.6 Deployment Plan 63
9.7 Tentative Timeline 63
9.8 Expected Final Deliverables 63

10 Conclusion **65**

A Proposal Action Register **67**

B Detailed Requirements **71**

C Detailed Test Cases **73**

D Repository and Contribution Evidence **77**

E Additional Design Diagrams **79**

F Meeting and Progress Evidence **81**

Bibliography **83**

List of Figures

4.1 Use case diagram of the GeoCrop platform 20

5.1 GeoCrop system architecture. The browser also reads PMTiles and uploads imagery directly to object storage using short-lived signed URLs; the tile service looks up layer styles from the API. 29
5.2 Domain model 31
5.3 Sequence of an upload and analysis 33
5.4 Job state machine 33

6.1 Crop-type experiment design 35
6.2 Run 2: eleven models on the same AOI1 test blocks (axis starts at 0.55; majority class 0.199 not shown) 40
6.3 Benchmark, improved inputs and neighbour vote in both areas (axis starts at 0.6) 42
6.4 Data per screen: prototype against tile-based delivery (about 130 times less) . 44
6.5 Home page with live figures from the AOI1 results 44
6.6 Live Map workspace: layer panel, crop-type layer over Sentinel-2 imagery, crop filter and Live map panel (tiles loaded and latency) 45
6.7 Analytics Dashboard: statistics from PostGIS and measured model accuracy . 45
6.8 System Status: health of every service and the architecture map 46

E.1 Deployment view of the Docker Compose stack 79
E.2 Activity of a model run inside the worker 79
E.3 Analyze Imagery: drag-and-drop upload, contextual data and model selection 80
E.4 Models page: registered models, status, accuracy and F1 per class 80

List of Tables

1.1 GeoCrop modules and responsibilities . . . 4
1.2 Stakeholders and user classes . . . 5

2.1 Summary of literature and gaps for GeoCrop . . . 8
2.2 Comparison of existing solutions with GeoCrop on project-relevant criteria . 10
2.3 Mapping of GeoCrop components to complex computing problem attributes . 11
2.4 How research findings shaped design decisions . . . 12

3.1 Proposal action register (summary; full register with evidence in Appendix A) 15
3.2 Updated iteration plan (replaces the phase plan presented in the proposal slides) 16
3.3 Work breakdown and ownership . . . 17
3.4 Planned phases (proposal slides) against actual progress . . . 17

4.1 Use cases . . . 19
4.2 Detailed use case UC-05: Upload imagery and run analysis . . . 21
4.3 Detailed use case UC-01 / UC-02: Browse layers and inspect values . . . 21
4.4 Detailed use case UC-11: Add or update an analysis model . . . 22
4.5 Acceptance criteria for important requirements . . . 25
4.6 Traceability of key requirements to modules, design, implementation and tests 26

5.1 Architectural components . . . 28
5.2 Main REST interfaces . . . 29
5.3 Key architecture decisions . . . 30
5.4 Design evolution since the proposal . . . 31
5.5 Database tables . . . 32

6.1 Use cases and features selected for Iteration 1 . . . 36
6.2 Platform code by component (lines, including configuration and documentation) . . . 36
6.3 Completed features at mid-term . . . 37
6.4 Important files, classes and functions . . . 38
6.5 Progress over three runs on AOI1 (all test pixels) . . . 39
6.6 Accuracy against cost (run 2, AOI1; single-thread CPU) . . . 40
6.7 Improvement ladder (inputs per month; AOI1 validation change; AOI2 scores) 41

6.8 Final results on both areas (all test pixels) . . . 41
6.9 GeoCrop crop-type results next to published studies (context, not a ranking) . 42
6.10 Tools, frameworks and technologies . . . 43
6.11 Rendering benchmark on AOI1 (crop-type layer, zoom 12) . . . 44
6.12 Measured resource requirements of the complete workflow (AOI1, one fresh process per stage) . . . 47

7.1 Revised evaluation criteria and status at mid-term . . . 50
7.2 Test levels and techniques . . . 51
7.3 Test cases (details in Appendix C) . . . 51
7.4 Test results . . . 52
7.5 Defects found during Iteration 1 and corrective actions . . . 53

8.1 Individual responsibilities . . . 56
8.2 Milestones . . . 57
8.3 External components, data and licences . . . 58
8.4 Risks and mitigation . . . 59

9.1 Remaining modules and features . . . 61
9.2 Tentative timeline . . . 63

A.1 Complete proposal action register . . . 67

B.1 Remaining use cases in brief . . . 71
B.2 Requirement priority and status . . . 72

C.1 Detailed test cases . . . 73
C.2 AOI1 per-class results (improved pipeline with neighbour vote; test pixels) . . 74
C.3 AOI2 per-class results (improved pipeline with neighbour vote) . . . 75
C.4 Fair reporting of AOI1 accuracy by subset of test pixels (accuracy % / macro-F1) . . . 75

D.1 Platform repository history (geocrop-platform, branch main) . . . 77
D.2 Prototype repository history (proposal stage) . . . 77

F.1 Progress evidence from dated project artefacts . . . 81
F.2 Supervisor meeting record (to be completed and signed by the team) . . . 82

Students' Submission

We certify that this report is submitted as the FYP-1 mid-report for the project titled *GeoCrop: Automated Crop Monitoring and Forecasting Toolkit*.

Student 1: Huzaifa Bin Khalid (22I-2332) Signature: _____

Student 2: Taha Bashir Awan (23I-0824) Signature: _____

Student 3: Saneed Ullah (23I-2568) Signature: _____

Anti-Plagiarism Statement

The students certify that this report is their own work and that all sources used in preparing the report have been appropriately acknowledged and referenced.

Authors' Declaration

We declare that the work documented in this report represents the work completed by the project team up to the FYP-1 mid-report stage. Contributions, tools, external components, and relevant sources are disclosed where applicable, including the use of an AI coding assistant during platform implementation (Section 8.7).

Abstract

Agricultural crop surveys still depend on field visits, manual crop identification and hand-drawn GIS boundaries, which makes them slow, costly and hard to repeat over large regions. GeoCrop is a satellite-to-GIS toolkit that learns crop classes from Sentinel-2 imagery and USDA Cropland Data Layer (CDL) labels, maps cropland and crop types, derives field regions and area, and will add yield estimation, all presented in a fast Web GIS portal. The research gap is that existing tools and studies solve crop classification, field mapping or yield prediction in isolation, and that research prototypes rarely remain usable once many images and model layers must be displayed. The project follows an iterative plan with two parallel tracks. In Iteration 1 the architecture track replaced the proposal-stage prototype, which sent a 43.3 MB GeoJSON of 101,435 polygons to the browser, with a tile-based platform: Cloud Optimized GeoTIFF results served by TiTiler, PMTiles vector layers, a React and MapLibre client, a FastAPI service, PostgreSQL with PostGIS, Celery workers and S3-compatible storage, all run with Docker Compose. One screen of the same area now costs 337 KB, cold map tiles return in 47 ms (median) and cached tiles in 1 ms, and an uploaded Sentinel-2 image is processed by both crop models and published to the map in 57 s. A manifest-based model registry lets new models plug in without changes to the API, database or interface. The model track evaluated eleven crop-type models on two study areas with a spatial block test; the proposed LightGBM pipeline with neighbourhood inputs and a 3×3 neighbour vote reaches 76.1% accuracy (macro-F1 0.686) in eastern Washington and 92.2% (macro-F1 0.803) in North Dakota, while staying about 2 MB and CPU-only. Validation so far covers registry unit tests, end-to-end API and browser tests, a rendering benchmark and the model evaluation protocol. FYP-2 will integrate the selected crop-type pipeline into the platform, add field-boundary and yield models, run cross-area tests and prepare deployment.

Acknowledgments

The project team acknowledges the guidance and support received from the project supervisor, Mr. Muhammad Aadil Ur Rehman, the external co-supervisor, Mr. Obaid Ur Rehman (Institute of Space Technology), the evaluation panel whose proposal feedback shaped this iteration, and the faculty of the FAST School of Computing. We also acknowledge the open data and open-source communities behind Sentinel-2, the USDA Cropland Data Layer and the geospatial software used in this project.

Executive Summary

Project and motivation. Agriculture departments need to know what is planted, where, over how much area, and how much may be harvested. Today this comes from physical surveys followed by manual GIS digitisation, which is time-consuming, resource-intensive and difficult to scale. GeoCrop automates the chain from satellite imagery to crop maps, field regions, area statistics and, in FYP-2, yield estimates, and presents them in a Web GIS portal (SDG 2, Zero Hunger).

Scope. The FYP builds a research-grade but usable prototype on open data: Sentinel-2 multispectral imagery as input, the USDA Cropland Data Layer (CDL) as training labels for eight U.S. areas of interest (AOIs), and USDA NASS statistics for yield. Production-scale national deployment, cadastral mapping and disease detection are out of scope.

Research findings. Two lines of investigation informed the design. On the platform side, profiling the proposal prototype showed that its crop-type result for one AOI was a 43.3 MB GeoJSON with 101,435 polygons (about 750,000 vertices), parsed three times and drawn as SVG, so loading and zooming were slow and would not scale to more images or models. Keeping model output as rasters and sending only the map tiles in view removes this cost. On the model side, eleven crop-type models were compared on AOI1 (eastern Washington, 14 classes) and AOI2 (North Dakota, 9 classes) using 12 monthly Sentinel-2 composites of 2022 and a spatial block split. A time series raised AOI1 accuracy from 26.7% (one date) to 75.2%; the model family mattered little (eight models within 2 points); neighbourhood inputs and a 3×3 vote added about 2 points; and 89% of the remaining errors sit on field-edge pixels. A small LightGBM model (about 2 MB, CPU-only) was proposed instead of the much slower Presto foundation model. The proposal was approved with minor modifications; the panel asked for clearer goals, evaluation criteria that measure performance, light weight, usability and integration, and a feasibility analysis of the whole workflow. All three are addressed: goals and criteria were restated with measurable targets, and every workflow stage was measured, showing that the baseline crop-type model (3.1 GB RAM) is the bottleneck that LightGBM (549 MB) removes.

Current implementation. Iteration 1 delivered a working platform with seven containerised services: a React and MapLibre Web GIS client, Nginx with a tile cache, a FastAPI service, a TiTiler tile service, Celery workers, PostgreSQL with PostGIS and MinIO object storage. It supports browsing and filtering layers, inspecting values at a point, field search, dashboard

statistics, uploading Sentinel-2 imagery, background model runs with live progress, a model catalogue and system health. A model registry runs the current cropland and crop-type models and lets new models plug in by folder and manifest.

Validation status. Registry unit tests pass (5/5), including a test that adds a model without touching other code. End-to-end tests covered every API route, upload to map in 57 s, crop filtering, field search and inspection in the browser. A benchmark measured 337 KB per screen against 43.3 MB, cold tiles at 47 ms (median) and cached tiles at 1 ms. Crop-type accuracy is reported against CDL labels on spatially separated test blocks, with confidence intervals.

Risks. CDL labels are themselves a model output; results cover one year and two areas; the platform still runs the proposal-stage baseline model until the selected pipeline is integrated; yield needs reliable historical ground truth; and boundary accuracy is limited at 30 m.

Remaining work. Iteration 2 (end of FYP-1) integrates the LightGBM crop-type pipeline, adds the CDL confidence filter and a cross-area test, and supports multi-image upload. FYP-2 adds field-boundary and yield models, an end-to-end evaluation against the success criteria, hardening and deployment.

List of Abbreviations

AOI	Area of Interest
API	Application Programming Interface
CDL	Cropland Data Layer (USDA)
COG	Cloud Optimized GeoTIFF
CPU	Central Processing Unit
CRS	Coordinate Reference System
DEI	Diversity, Equity and Inclusion
ERD	Entity Relationship Diagram
F1	Harmonic mean of precision and recall
FR	Functional Requirement
FYP	Final Year Project
GDAL	Geospatial Data Abstraction Library
GEE	Google Earth Engine
GIS	Geographic Information System
JWT	JSON Web Token
LightGBM	Light Gradient Boosting Machine
MLP	Multi-Layer Perceptron
MVT	Mapbox Vector Tile
NASS	National Agricultural Statistics Service (USDA)
NDVI	Normalized Difference Vegetation Index
NFR	Non-Functional Requirement
NIR	Near-Infrared
OSM	OpenStreetMap
PostGIS	Spatial extension for PostgreSQL
RAM	Random Access Memory
RF	Random Forest
S3	Simple Storage Service (object storage API)

SDG	Sustainable Development Goal
SUS	System Usability Scale
SWIR	Short-Wave Infrared
TC	Test Case
UC	Use Case
USDA	United States Department of Agriculture
XYZ	Web map tile addressing scheme (zoom, column, row)

Chapter 1

Introduction and Project Vision

1.1 Project Overview

GeoCrop is a satellite-based agricultural intelligence toolkit. It takes multispectral satellite imagery of an agricultural region, identifies which land is cropland, classifies the crop growing in each 30 m pixel, groups pixels into field regions with area, and (in FYP-2) estimates yield. The results are published as map layers and statistics in a Web GIS portal, where users can switch layers on and off, filter by crop, search fields, inspect values at any location and read crop-area summaries.

The system is built on open data. Sentinel-2 [26] provides the imagery, the USDA Cropland Data Layer (CDL) [27, 1, 2] provides labelled crop maps for training and evaluation over eight U.S. areas of interest (AOIs), and USDA NASS statistics will provide historical yield. GeoCrop is designed as an academic, department-oriented prototype whose models, evaluation method and outputs are transparent, and whose software architecture can carry many images and many models without slowing down.

1.2 Problem Statement

The current crop-survey workflow relies on manual field inspection, crop identification, boundary digitisation, area calculation and GIS data entry. This process is time-consuming, needs significant human resources, is difficult to repeat frequently over large regions, and can introduce inconsistencies in crop classification and field mapping. Satellite imagery observes agricultural land frequently and with geographic reference, but turning those observations into crop maps, crop types, field regions, acreage and expected yield still requires substantial processing and expertise, and the results must be delivered in a form that planners can use interactively over large areas.

There is therefore a need for an integrated system that learns crop classes from labelled geospatial data, analyses new satellite imagery, distinguishes cropland from other land, clas-

sifies supported crop types, derives field regions and area, predicts yield from suitable spatio-temporal features, and presents the results through a responsive Web GIS portal that remains fast as the number of images and model layers grows.

1.3 Problem Context and Motivation

Agricultural departments, field survey teams, GIS analysts and planners need timely information about what is planted, where, and over how much area. In the current workflow (physical survey, crop and non-crop identification, crop-type identification, boundary drawing, area and coordinate calculation, GIS entry, separate yield assessment) every step is manual. This is slow, needs many people, is hard to scale across large areas, and can introduce errors.

Open Earth-observation missions now give repeat multispectral coverage, and machine learning has shown that crop classes can be learned from spectral and temporal patterns [5, 7, 3]. Research in Pakistan confirms that Sentinel-2 can identify crops in smallholder settings, but still depended on thousands of manually surveyed field boundaries [5]. In the United States, the CDL offers large labelled coverage that has been used to train classifiers [2, 4]. GeoCrop uses this labelled coverage to develop and evaluate the methodology before adaptation to local data.

The motivation is also practical. A crop map is only useful if people can work with it. The proposal-stage prototype showed that a single AOI's crop-type result, shipped as vector polygons, already made the map slow to load and zoom. A system meant for many images and several models (cropland, crop type, boundaries, yield) has to be designed for performance from the start. Better agricultural monitoring supports informed crop planning and food-security decisions (SDG 2, Zero Hunger).

1.4 Project Vision

GeoCrop aims to give agricultural organisations a single toolkit that turns new satellite imagery into trustworthy, map-ready crop intelligence in minutes: where the cropland is, what is growing, how large each crop area is, and what yield to expect. Results should be honest about their accuracy, reproducible, and fast to explore on a map, and new models should plug into the same platform as research progresses.

1.5 Project Objectives

After the proposal defence the panel asked the team to clarify its goals (P1, Section 3.1). The goals are now stated as follows. GeoCrop's primary goal is a *light, fast and integrated* platform: one that turns an uploaded Sentinel-2 image into crop layers on a map without heavy hardware, and that stays usable as images and models grow. Its research goal is crop-type models that are accurate under a fair test and cheap enough to serve. Three terms from

the proposal are defined precisely:

- **Crop mapping**: per-pixel cropland and crop-type maps at 30 m for supported AOIs, scored against the CDL on spatially separated test blocks.
- **GIS integration**: every result is a georeferenced layer in one catalogue, viewable, filterable and comparable on the same map as the imagery and other layers.
- **Real-time estimation**: *near* real time on demand. An AOI-sized image is analysed and published in about a minute (57 s measured), and a map screen loads in well under a second. Continuous streaming of new imagery is not a goal.

Each objective below has a measurable target that is checked in Chapter 7.

O1. Build a spatially aligned Sentinel-2 and CDL training dataset for at least two AOIs, with a reproducible preparation pipeline.

O2. Develop and evaluate cropland and crop-type models using a spatially separated test set, reporting accuracy, macro-F1 and per-class scores with confidence intervals, and compare them with published results.

O3. Derive field regions and area from model output and publish them as GIS layers with geographic coordinates; add a dedicated field-boundary model in FYP-2.

O4. Develop a yield-estimation module evaluated with R^2, RMSE and MAE against USDA NASS records at the spatial scale for which ground truth exists.

O5. Deliver a Web GIS portal that shows one screen of results using under 1 MB of data, returns cold map tiles in under 300 ms and cached tiles in under 50 ms, and keeps several layers visible at once.

O6. Provide a processing pipeline in which an uploaded image is validated, analysed by the selected models and published to the map automatically, and in which a new model can be added without changing the API, database schema or user interface.

1.6 Project Scope

Included.

- Satellite data preparation: Sentinel-2 composites, band selection, reprojection to the CDL grid, vegetation indices and monthly time series.
- CDL-based label preparation for the supplied AOIs, including rare-class grouping.
- Cropland/non-cropland and multi-class crop-type mapping with model comparison.

- Field-region derivation, area statistics and vector map layers; a field-boundary model in FYP-2.

- Yield estimation using satellite features and USDA NASS history (FYP-2).

- A Web GIS platform: upload and processing jobs, layer catalogue, tile-based map, dashboard, model catalogue, system status, and user accounts.

Excluded. Building or operating satellites; legal or cadastral ownership mapping; crop disease diagnosis; real-time farm machinery integration; production-scale national deployment with service-level guarantees; and claims of ground-truth accuracy beyond agreement with the reference labels.

1.7 Project Modules

Table 1.1: GeoCrop modules and responsibilities

Module	Responsibility	Status
M1 Data acquisition and preprocessing	Export Sentinel-2 composites, align them to the CDL grid (EPSG:5070, 30 m), build monthly time series and indices, convert results to Cloud Optimized GeoTIFF.	Working for AOI1, AOI2
M2 Crop mapping	Cropland/non-cropland and crop-type models; evaluation and model selection.	Baseline in platform; research pipeline selected
M3 Field regions, boundaries and area	Polygonise model output, compute area, build vector tiles; field-boundary model later.	Field regions working; model in FYP-2
M4 Yield estimation	Regression on satellite features and NASS yield history.	FYP-2 (registered placeholder)
M5 Processing pipeline	Upload, validation, preprocessing, background model runs, progress tracking.	Working
M6 Model registry	Plug-in contract, manifests, dependency resolution, model catalogue.	Working
M7 Catalogue, storage and statistics	PostGIS catalogue of areas, runs, layers, statistics and fields; S3 object storage.	Working
M8 Web GIS portal	Home, Live Map, Dashboard, Analyze Imagery, Models, System Status, accounts.	Working

Table 1.2: Stakeholders and user classes

Stakeholder / user class	Characteristics and needs
Agriculture Department	Needs regional crop area and expected production for planning; values accuracy and transparency.
Field survey teams	Today collect crop and boundary data manually; benefit from fewer field visits and targeted checks.
GIS analysts	Technical users; need georeferenced layers, coordinates, area and exportable data.
Agricultural planners and decision makers	Non-technical; need clear maps, legends and summary statistics.
Farmers	Indirect beneficiaries of better planning and advisory services.
Industry / startup partner	External collaborator interested in a scalable, light system.
Platform developers and ML engineers	Add and update models, monitor the system, run evaluations.

1.8 Stakeholders and User Classes

1.9 Success Criteria

- **Crop mapping:** crop-type accuracy and macro-F1 reported on spatially separated test blocks with 95% intervals; results comparable in order of magnitude to published CDL-based studies for similar regions (for example 94% South Dakota and 83% California in [3]).
- **Boundaries and area:** spatial overlap (IoU) and area error against reference data, and acreage within a stated tolerance of USDA statistics (FYP-2).
- **Yield:** R^2, RMSE and MAE on held-out years or counties (FYP-2).
- **Performance:** first visible result under 1 s, under 1 MB per screen, cold tiles under 300 ms, cached tiles under 50 ms, eight or more layers visible.
- **Pipeline:** an uploaded image reaches the map without manual steps, with readable errors for invalid input; a new model is added by folder and manifest only.
- **Lightweight:** the full workflow runs on one laptop-class host; the selected crop-type model is CPU-only and small; peak memory per stage is measured.
- **Usability:** non-technical users can find, filter and read results using legends and plain-language labels; measured with task completion and a questionnaire score in a user study.
- **Integration:** several layers from different models and images can be used together on one map without loading whole datasets.

1.10 Constraints and Assumptions

- CDL labels are a classified product, not field survey data; scores measure agreement with the CDL [1].
- Sentinel-2 at 10–20 m, resampled to the 30 m CDL grid, limits boundary precision for small fields.
- Data for 2022 is used so that imagery and labels refer to the same season.
- Yield ground truth (USDA NASS) is available at county level, which bounds the spatial scale of yield evaluation.
- Compute is limited to student laptops and free cloud notebooks, and the panel named a lightweight system as a main concern, so CPU-only inference is preferred and resource use is measured for every stage (Section 6.9.1).
- Uploaded imagery is assumed to be a georeferenced Sentinel-2 surface-reflectance stack with the documented bands; other formats are rejected with a clear message.
- The platform runs on Docker Compose on a single host for development and demonstration.

1.11 Ethical, Legal and DEI Considerations

- **Honest reporting:** predictions are never presented as ground truth. The portal shows model accuracy, its evaluation protocol and limitations; placeholder models are labelled "coming soon" and never produce data.
- **Fairness across classes and regions:** per-class scores and per-area results are reported so that unequal performance (for example on small or rare crops) is visible.
- **Licensing:** Sentinel-2 data (Copernicus open licence), CDL and NASS data (U.S. public domain), OpenStreetMap basemap (ODbL attribution shown) and open-source libraries are used within their terms and acknowledged.
- **Privacy:** uploaded imagery and results belong to the uploading account; passwords are hashed; startup-provided data will be protected and not published.
- **Accessibility and inclusion:** the interface uses clear legends, plain-language labels, keyboard-accessible controls and readable colour palettes so that technical and non-technical staff can use it.
- **Disclosure:** external datasets, models, libraries and AI-assisted work are disclosed (Sections 8.7 and 8.8).

Chapter 2

Research and Existing Solutions

2.1 Research Method / Approach

Knowledge was built along four lines.

1. **Literature review.** Studies on CDL-based crop classification, crop mapping in Pakistan, field-boundary extraction, yield prediction and time-series deep learning were reviewed (Table 2.1). Eight further papers were read for the crop-type comparison (Section 6.7.6).

2. **Data analysis.** The supplied CDL rasters and AOI shapefiles were analysed for class distribution, field layout and edge pixels; Sentinel-2 composites were exported with Google Earth Engine and aligned to the CDL grid.

3. **Controlled experiments.** Crop-type models were compared under one protocol: 12 monthly composites of 2022, a spatial block split with a buffer, normalisation on training data only, and all choices made on validation data (Chapter 6).

4. **Engineering investigation.** The proposal-stage prototype was profiled to find its performance limits, candidate geospatial technologies were compared, and the chosen design was benchmarked on the real AOI1 data.

2.2 Existing Solutions / Products

USDA CropScape / CDL provides annual georeferenced crop maps and acreage statistics for the United States. It is an already-classified product and does not analyse new imagery supplied by a user, estimate yield, or provide a configurable workflow.

QGIS and ArcGIS-type GIS tools visualise geospatial data, edit polygons, calculate area and manage coordinates. Crop identification and yield prediction still require separate data, models and expert workflows.

DSSAT crop models simulate crop growth and estimate yield from agronomic and environmental inputs. They do not classify crops from satellite imagery or extract field regions.

OneSoil and similar precision-agriculture platforms combine satellite imagery, vegetation indices and field monitoring in a map interface [29]. They are commercial and closed, so models, training data and accuracy are not transparent or adaptable to a department's data.

Prior FYPs typically address one task: crop classification (model accuracy only), GIS or field mapping (classification supplied manually), or yield prediction (no automatic crop mapping or boundaries).

Research literature. Table 2.1 summarises the most relevant studies.

Table 2.1: Summary of literature and gaps for GeoCrop

No.	Paper / year	Focus	Key contribution	Gap for GeoCrop
1	Lin et al. 2022 [1]	CDL refinement	Spatial-temporal decision tree flags and corrects questionable CDL pixels; pixel and county validation.	Shows CDL label noise at edges; no new-image classification or yield.
2	Hao et al. 2020 [2]	CDL transfer learning	High-confidence CDL pixels with Landsat/Sentinel NDVI time series train Random Forests (OA 86–98%).	Classification only; no GIS boundary or yield workflow.
3	Tran et al. 2022 [3]	10 m crop type from CDL	Monthly Sentinel-2 composites and good-quality CDL samples; OA 94% (South Dakota), 83% (California).	Random sample split and quality filtering; no platform.
4	Wang et al. 2020 [4]	Historical mapping	CDL labels with Landsat reconstruct 20 years of corn and soybean in the Midwest.	Two crops only; no integrated portal.
5	Khan et al. 2023 [5]	Punjab crop ID	Stacked LSTM on Sentinel-2 time series for rice, wheat, sugarcane.	Relies on GPS ground surveys; no boundaries or yield.
6	Latif et al. 2023 [6]	Pakistan cropland	Multi-date Sentinel-2 and ML on Google Earth Engine for cropland extent.	Cropland only; no crop types or yield.

Continued on next page

No.	Paper / year	Focus	Key contribution	Gap for GeoCrop
7	Muqaddas et al. 2024 [7]	Punjab sugarcane	Deep learning on Sentinel-2 indices; cross-district tests.	Single crop; no integrated workflow.
8	Edri et al. 2026 [8]	Field boundaries	SAM, U-Net, DeepLabV3, SegFormer on time series refine field polygons.	Assumes a parcel registry; no crop type or yield.
9	Desloires et al. 2023 [9]	Corn yield	Sentinel-2 and environmental features; out-of-year evaluation.	Assumes known fields and yield labels.
10	Xiao et al. 2024 [10]	Wheat yield	Attention 1D CNN on Sentinel-2 time series.	Single crop; no mapping.
11	You et al. 2017 [11]	County yield	Deep Gaussian process on remote-sensing histograms.	County level; no field mapping.
12	Zhang et al. 2025 [12]	Within-season U.S. crop type	Transformer on HLS time series, CDL labels, 37 crop types.	Large model (about 17 M parameters); research code only.
13	Li et al. 2024 [13]	In-season CDL	Trusted pixels from CDL history, Random Forest; corn/soy F1 0.85–0.97.	No user-facing workflow.
14	Tseng et al. 2023 [16]	Presto	Light pre-trained transformer for pixel time series.	Benefit shown mostly with few labels.
15	Szwarcman et al. 2024 [14]	Prithvi-EO-2.0	300–600 M parameter foundation model; crop segmentation mIoU up to 50.7.	Too heavy for a light CPU system.

2.3 Comparison of Existing Solutions

2.4 Limitations and Gaps

- **Fragmented workflows.** Crop classification, field mapping and yield prediction are usually separate tools or studies; users must combine them by hand.
- **Dependence on manual surveys.** Local studies rely on GPS field surveys [5]; CDL supervision reduces the labels needed for early development.
- **Optimistic evaluation.** Many studies use random pixel or sample splits. Our own

Table 2.2: Comparison of existing solutions with GeoCrop on project-relevant criteria

Solution	Classifies new imagery	Field regions / area	Yield	Web map	Open / trans-parent	Scales to many layers
USDA CropScape / CDL	–	partial	–	✓	✓	✓
QGIS / ArcGIS	–	✓	–	partial	partial	partial
DSSAT	–	–	✓	–	✓	–
OneSoil-type platforms	✓	✓	partial	✓	–	✓
Prior FYPs	partial	partial	partial	partial	✓	–
GeoCrop proposal prototype	✓	partial	–	✓	✓	–
GeoCrop (this work)	✓	✓	FYP-2	✓	✓	✓

test showed that a random split leaks neighbouring pixels (macro-F1 0.396 random against 0.147 with blocks on the same model), so published numbers are not always comparable.

- **Prototypes that do not scale.** Research outputs are often shipped as large vector files. The proposal prototype needed 43.3 MB and 101,435 SVG polygons for one AOI's crop-type layer, which made the map slow and would not scale to several images and models.
- **Heavy models.** Foundation models are accurate but need hundreds of megabytes or GPUs, which conflicts with a light, affordable system.
- **Label uncertainty.** CDL contains mixed-pixel and boundary errors [1], which must be accounted for in training and reporting.

2.5 Proposed Solution

GeoCrop connects the tasks into one satellite-to-GIS workflow and builds it on an architecture designed for performance and change:

- **Raster-first results served as tiles.** Model outputs are stored as Cloud Optimized GeoTIFFs and served as small map tiles by TiTiler; vector field regions are packed as PMTiles. The browser downloads only what is on screen.
- **A plug-in model registry.** Cropland, crop-type, boundary and yield models share one contract; dependencies run in order; new models are added by folder and manifest.
- **An asynchronous processing pipeline.** Uploads go straight to object storage; Celery workers validate, preprocess, run models and publish each layer with statistics.

- **A transparent, light crop-type model.** A LightGBM pipeline on monthly Sentinel-2 composites with neighbourhood inputs and a neighbour vote, evaluated with a spatial block split and reported with intervals and per-class scores.
- **A usable portal.** Layer panel, crop filter, field search, point inspection, dashboard, model catalogue and system health in one interface.

2.6 Complex Computing Problem

Under the Seoul Accord definition [28], GeoCrop is a complex computing problem: it has no obvious solution, combines several knowledge areas and has interdependent parts.

Table 2.3: Mapping of GeoCrop components to complex computing problem attributes

Component	Attributes and evidence
Crop mapping and classification	*Conflicting requirements* (accuracy against model size, speed and cost); *depth of analysis* (no single best model: eleven compared, label noise, edge pixels); *depth of knowledge* (ML, multispectral remote sensing, phenology); *interdependence* (preprocessing and label alignment drive accuracy).
Field regions, boundaries and area	*Depth of knowledge* (CRS, raster/vector processing, area in projected units); *interdependence* (crop-map errors propagate to polygons and acreage); *consequences* (wrong acreage misleads planning).
Yield estimation	*Depth of analysis* (temporal features, scale of ground truth); *consequences* (production forecasts); *conflicting requirements* (granularity against available labels).
Platform and rendering	*Conflicting requirements* (detail against speed and data volume); *depth of knowledge* (COG, tiling, vector tiles, spatial databases, distributed jobs); *stakeholder needs* (technical and non-technical users); *interdependence* (every model output must become a correct, fast layer).

2.7 Technical Challenges

- **Rendering performance.** Pixel-level predictions vectorised into polygons produce very large files (101,435 polygons for one AOI). Displaying many images and layers requires tiling, caching and immutable versioned URLs.
- **Spatial alignment.** Sentinel-2 (10–20 m, UTM) must be reprojected exactly onto the CDL grid (EPSG:5070, 30 m); web maps use EPSG:3857.
- **Label noise and edge pixels.** In AOI1, 47% of test pixels touch another class and hold 89% of the errors; CDL itself is noisy there.

- **Evaluation leakage.** Random pixel splits overstate accuracy; spatial blocks with a buffer are needed.
- **Cloud gaps.** Missing monthly observations must be filled consistently.
- **Model size and speed.** The default Random Forest is 328 MB; the baseline crop-type model in the platform is 1.5 GB and must be loaded per run; Presto is 660 times slower than an MLP.
- **Integration.** Model outputs, statistics, styles and files must stay consistent across the database, object storage, tile service and client.
- **Operations.** Container start-up races, service discovery after rebuilds and browser caching had to be handled for a reliable demonstration.

2.8 Research Findings and Design Decisions

Table 2.4: How research findings shaped design decisions

Finding	Decision
One AOI's crop-type GeoJSON was 43.3 MB with 101,435 polygons; parsing alone took about 1 s.	Keep results as rasters (COG); serve XYZ tiles with TiTiler; vectors only as PMTiles.
Most of the prototype polygons (85%) were single-pixel squares.	Raster is the source of truth; polygons are derived, filtered by minimum size and simplified per zoom.
A time series lifted AOI1 accuracy from 26.7% to 75.2%.	Use 12 monthly composites of the label year as model input.
Eight model families finished within 2 points; logistic regression 4.8 points behind.	Choose the model on cost: LightGBM (about 2 MB, CPU); keep Random Forest as baseline.
Presto matched simple models but was about 660 times slower per pixel.	Reverse the earlier Presto recommendation for the U.S./CDL case.
Random pixel splits leak neighbours (macro-F1 0.396 against 0.147).	Spatial 100×100 block split, 3-pixel buffer, choices on validation only.
89% of errors are on field-edge pixels.	Neighbourhood inputs and a 3×3 vote now; field-level classification later.
Models and outputs will keep changing (cropland, crop type, boundary, yield).	Manifest-based model registry with a dependency graph and generic layer catalogue.
Heavy work (image conversion, inference) takes tens of seconds.	Asynchronous Celery jobs with progress; direct-to-storage uploads.

Chapter 3

Proposal Feedback and Iteration Plan

3.1 Proposal Feedback Summary

The proposal defence result was sent by the FYP Committee on 14 September 2026. The decision for F26-155-GeoCrop was **Approved with Minor Modifications**. The panel's comments are summarised below; each is numbered P1–P7 and tracked in the action register.

P1. Clarify the intended goals. The proposal claimed crop mapping, GIS integration and real-time estimation without stating clearly what each meant.

P2. Improve the evaluation metrics. The proposed criteria did not measure the actual challenges of the project.

P3. Measure the real challenges: computational performance, a lightweight system and usability. The panel stated that the evaluation criteria were wrong, and that usability, performance and integration are the main concerns.

P4. Analyse feasibility. The complexity and resource requirements of the complete workflow had not been analysed beforehand.

P5. Do not run everything in a single flow. The computational and space requirements are high enough that running the entire process in a single flow may not be practical.

P6. Integration of huge datasets. Large datasets are not easy to load and use together.

P7. Work on presentation skills.

The panel also asked the team to incorporate all changes it suggested. These comments set the agenda for Iteration 1. The architecture track answered P3–P6 by redesigning the platform around performance, light weight and integration, and measuring each one. The model track answered P2–P4 by changing the evaluation protocol and selecting the model on cost as well as accuracy.

The team's own review after the defence found further issues: the prototype could not run models on uploaded images, the first evaluation used an optimistic random pixel split, models would keep changing during the project, and the iteration plan in the proposal slides no longer matched the order in which the work had to be done. These items are recorded separately in the register, with their source.

3.2 Proposal Action Register

3.3 Iteration Strategy

Development is iterative and time-boxed. Each iteration ends with a working, demonstrable increment, a short evaluation, and an updated plan. Because GeoCrop has two kinds of risk, work proceeds on two parallel tracks that meet at the end of each iteration:

- **Architecture track** (platform): data flow, storage, processing, map delivery and interface. Risk: performance and integration.
- **Model track** (research): data preparation, model comparison and evaluation. Risk: accuracy, generalisation and cost.

The two tracks share one contract: a model is a plug-in that reads an aligned image and writes a georeferenced raster, so research results can move into the platform without redesign. Decisions are made on measured evidence (benchmarks for the platform, validation scores for models) and recorded in this report and the repository.

3.4 Iteration 1 Plan

Objectives. (1) Replace the prototype with a base architecture that stays fast as images and models grow, implemented and benchmarked on real data. (2) Find a crop-type model that is accurate under a fair test and light enough for the platform.

Selected features. Tile-based Live Map with layer panel, opacity, legends, crop filter, field search and point inspection; dashboard statistics from the database; upload and background processing with progress; model registry with the current cropland and crop-type models; system-status page. On the model side: monthly composites for AOI1 and AOI2, eleven-model comparison, improvement ladder, error analysis and comparison with published papers.

Deliverables. Platform repository with Docker Compose stack and documentation; benchmark results; crop-type results report; this mid-report.

Evidence. Benchmark tables, test results, screenshots (Section 6.9) and repository history (Appendix D).

Table 3.1: Proposal action register (summary; full register with evidence in Appendix A)

No.	Feedback / required change	Action taken	Status	Evidence
P1	Clarify the intended goals.	Objectives restated with measurable targets; "real-time" defined as an uploaded image on the map in about a minute; scope split by FYP phase.	Done	Sec. 1.5
P2, P3	Evaluation metrics must measure performance, light weight, usability and integration.	Evaluation criteria rewritten around these concerns, each with a metric and target. Performance, cost and integration measured; usability checked against requirements, user study planned.	Partly done	Table 7.1
P4	Analyse feasibility and resource requirements of the whole workflow.	Time, memory and storage measured for every stage; the baseline crop-type model (3.1 GB RAM) found to dominate, so it is replaced by LightGBM (549 MB).	Done	Table 6.12
P5	A single-flow process is not practical.	Workflow split into independent queued stages; results kept as compressed rasters; maps served tile by tile on demand.	Done	Sec. 6.9.1
P6	Huge datasets are hard to load and use together.	Only visible tiles are sent: 337 KB per screen instead of 43.3 MB; layers share one catalogue.	Done	Table 6.11
P7	Presentation skills.	Fixed demonstration script on the running system; each member presents their own track; rehearsals before evaluations.	In progress	Sec. 6.9
T1	Team review: uploaded images were never analysed.	Upload, validation, preprocessing and model runs as background jobs; 57 s upload-to-map.	Done	TC-11, Fig. 5.3
T2	Team review: random pixel split overstates accuracy.	Spatial block split with buffer; choices on validation; intervals over test blocks.	Done	Sec. 6.7.2
T3	Team review: models will keep changing.	Manifest-based model registry; add-a-model test.	Done	TC-03
T4	Team review: iteration plan in the slides outdated.	Two-track iteration plan (Sections 3.4, 3.5, Chapter 9).	Done	This chapter

Table 3.2: Updated iteration plan (replaces the phase plan presented in the proposal slides)

Iteration	Period	Focus
Iteration 1	Sep – mid-Oct 2026 (FYP-1, to mid-term)	Base architecture with implementation and performance benchmark; crop-type model evaluation and comparison.
Iteration 2	mid-Oct – Dec 2026 (FYP-1, to final)	Integrate the selected crop-type pipeline into the platform; CDL confidence rule; cross-area test; multi-image upload.
Iteration 3	Feb – Mar 2027 (FYP-2)	Field-boundary model and area validation; yield data and first yield model; more AOIs.
Iteration 4	Apr – May 2027 (FYP-2)	End-to-end evaluation, hardening, deployment and final documentation.

3.5 Iteration 2 Plan

Objectives. Bring the selected crop-type pipeline into the platform and strengthen the evaluation.

Selected features.

- Package the LightGBM pipeline (monthly composites, neighbourhood inputs, 3×3 vote) as a registry model and replace the baseline crop-type model in the platform.
- Monthly-composite ingestion in the processing pipeline.
- Fix one CDL confidence rule and report it next to the all-pixel score; compute the AOI2 interval and interior table.
- Cross-area test (train on one AOI, test on the other).
- Multi-image upload and an "all areas" map view.
- Measure end-to-end serving cost of the improved pipeline on CPU and repeat the workflow resource measurement (Table 6.12).
- Usability study: task-based sessions with a questionnaire score and task times (P3).
- Multi-layer test: eight or more layers visible at once (NFR-P3).

Deliverables. Updated platform release, evaluation addendum and FYP-1 final report. **Evidence.** Test results, benchmark re-run and model cards with stored metrics.

Table 3.3: Work breakdown and ownership

Work package	Tasks	Owner
WP1 Architecture and platform	Architecture design; performance analysis of the prototype; tile serving; storage and database design; processing pipeline; model registry; web portal; deployment configuration; benchmark.	Huzaifa Bin Khalid
WP2 Crop-type model evaluation	Monthly composite preparation; spatial block protocol; model comparison and cost measurement; improvement ladder; error analysis.	Saneed Ullah, Taha Bashir Awan
WP3 Comparison with literature	Reading and checking reference papers; like-for-like comparison and limits.	Saneed Ullah, Taha Bashir Awan
WP4 Integration (Iteration 2)	Packaging the selected model as a registry plug-in; end-to-end measurement.	Shared
WP5 Documentation and evaluation	Reports, demonstrations, final evaluation.	Shared

Table 3.4: Planned phases (proposal slides) against actual progress

Planned (slides)	Planned content	Actual at mid-term
Phase 1 (FYP-1) Research and data understanding	Literature review; CDL/AOI analysis; satellite data acquisition	Done. Literature extended with eight crop-type papers; monthly 2022 composites exported for AOI1 and AOI2.
Phase 2 (FYP-1) Core crop mapping pipeline	Preprocessing; training dataset; baseline model; initial GIS boundary extraction	Done for two AOIs. Eleven models compared; improved pipeline selected. Field regions derived from crop-type output as interim boundaries.
Phase 3 (FYP-2) Yield and platform integration	Yield module; Web GIS; system integration	Platform architecture and integration brought forward and done in FYP-1. Yield remains in FYP-2.
Phase 4 (FYP-2) Evaluation and finalisation	Hold-out tests; failure analysis; performance evaluation; deployment	Partly brought forward: spatial hold-out testing, error analysis and performance benchmark already done; final evaluation and deployment remain.

3.6 Work Breakdown and Responsibilities

3.7 Progress Against Plan

3.8 Delays, Deviations and Recovery Plan

- **Platform work brought forward (deviation).** The slides placed Web GIS and integration in FYP-2. Profiling showed the prototype would not scale, and the panel named performance, light weight, usability and integration as the main concerns (P3–P6), so the architecture was redesigned and implemented in Iteration 1. This reduces integration risk in FYP-2.
- **Technology change (deviation).** The Django and Leaflet prototype that sent GeoJSON was replaced by FastAPI, React with MapLibre, TiTiler, PMTiles, PostGIS, Celery and MinIO (Section 5.5).
- **Model recommendation reversed (deviation).** Early research notes recommended the Presto foundation model; the experiments did not support it for the U.S./CDL case, so a LightGBM pipeline is proposed.
- **Platform still runs the baseline crop-type model (delay).** The selected pipeline needs monthly composites at ingest; integration is the first item of Iteration 2.
- **Boundary model not started (delay).** Field regions derived from crop-type output act as the interim boundary layer; a boundary model is planned for Iteration 3.
- **Two of eight AOIs processed (delay).** Monthly composites exist for AOI1 and AOI2; the remaining AOIs will be exported for the cross-area and generalisation tests.
- **AI-assisted implementation (deviation from the stated GenAI boundary).** The proposal limited generative AI to documentation; during Iteration 1 an AI coding assistant was also used for parts of the platform implementation, under the team's design and review (Section 8.7).

Recovery. Iteration 2 starts with model integration, then the CDL confidence rule and the cross-area test; AOI exports run in the background on Google Earth Engine. The boundary and yield work in FYP-2 benefits from the registry and pipeline already in place, so new models only need a task and a manifest.

Chapter 4

Software Requirements

4.1 User Interactions / Use Cases

The main actors are the **Agricultural Analyst** (planners and department staff who explore and read results), the **GIS Analyst** (technical user who uploads imagery and checks layers), the **Platform Developer / ML Engineer** (adds and updates models, monitors the system) and the **Processing Worker** (system actor that runs background jobs).

Table 4.1: Use cases

ID	Use case	Primary actor
UC-01	Browse result layers on the live map	Agricultural Analyst, GIS Analyst
UC-02	Inspect values at a location	Agricultural Analyst, GIS Analyst
UC-03	Filter by crop and search field regions	Agricultural Analyst
UC-04	View crop statistics on the dashboard	Agricultural Analyst
UC-05	Upload imagery and run analysis	GIS Analyst
UC-06	Track a processing job	GIS Analyst
UC-07	Re-analyse an existing image	GIS Analyst
UC-08	View models and their accuracy	All users
UC-09	Create an account and sign in	All users
UC-10	Monitor system status	Platform Developer
UC-11	Add or update an analysis model	Platform Developer / ML Engineer
UC-12	Process an image in the background	Processing Worker (system)

4.2 Use Case Diagram

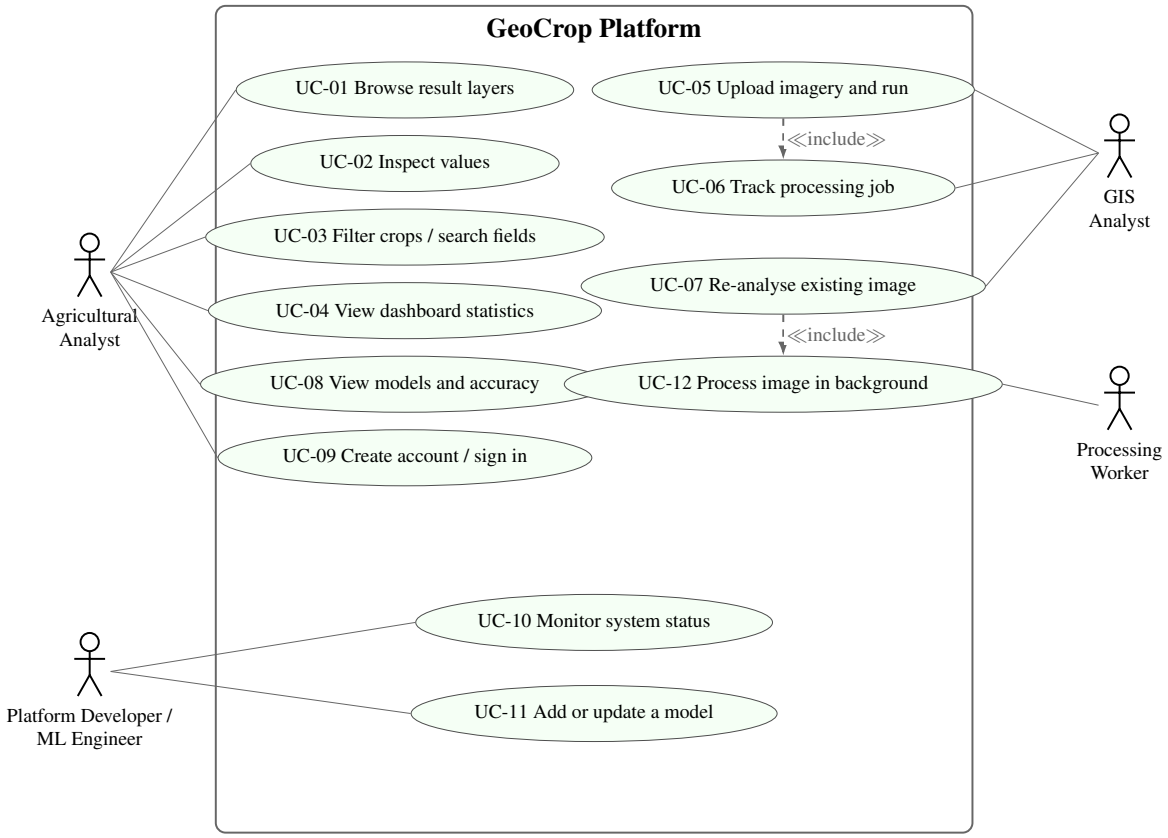


Figure 4.1: Use case diagram of the GeoCrop platform

4.3 Detailed Use Cases

Three central use cases are given here; the remaining ones are in Appendix B.

4.4 Functional Requirements

4.4.1 Module 1: Processing Pipeline (Ingestion and Jobs)

FR-ING-01 The system shall accept GeoTIFF uploads up to a configured size limit.

FR-ING-02 The system shall upload files directly to object storage through signed URLs, with an API fallback.

FR-ING-03 The system shall validate that an image is readable, georeferenced and has a supported band count, and report the reason when it is not.

FR-ING-04 The system shall reproject images to the model grid (EPSG:5070, 30 m) and convert them to Cloud Optimized GeoTIFF.

FR-ING-05 The system shall run analysis as background jobs and record per-step progress.

Table 4.2: Detailed use case UC-05: Upload imagery and run analysis

Field	Description
Actor	GIS Analyst
Preconditions	Platform services are running; the user has access (sign-in optional in the current configuration).
Trigger	User opens Analyze Imagery.
Main flow	1. User drops or selects a Sentinel-2 GeoTIFF and optionally enters location and capture date. 2. User selects analysis models; dependencies (crop type needs cropland) are resolved automatically. 3. System creates an upload record and returns a signed storage URL; the browser uploads the file directly to object storage. 4. System creates a job with planned steps and queues ingestion; user is taken to the Processing page. 5. Worker validates, reprojects to the model grid, converts to COG and publishes the image layer (UC-12). 6. Worker runs the selected models in dependency order and publishes each result layer with statistics. 7. User sees "Analysis complete" and opens the result on the Live Map.
Alternative flows	1a. File is not a GeoTIFF: rejected before upload with a message. 5a. No coordinate system or wrong band count: job fails with a readable reason. 6a. A model is a placeholder: its step is skipped and marked as not available.
Postconditions	New area, image layer, result layers, statistics and job history are stored.
Related requirements	FR-ING-01 to FR-ING-07, FR-REG-03, NFR-P4

Table 4.3: Detailed use case UC-01 / UC-02: Browse layers and inspect values

Field	Description
Actor	Agricultural Analyst, GIS Analyst
Preconditions	At least one area with published layers exists.
Main flow	1. User opens Live Map; the first area is selected and imagery and crop type are shown. 2. User toggles layers in the Layer panel, changes opacity and reads legends. 3. The map requests only the tiles in view; the Live map panel shows tiles loaded and latency. 4. User clicks the map; the system returns the value and class label of every visible raster layer and the field region under the cursor.
Alternative flows	2a. Area has no layers: an empty state explains how to create them. 4a. Point outside the data: "No data".
Postconditions	None (read-only).
Related requirements	FR-VIS-01 to FR-VIS-07, NFR-P1 to NFR-P3

Table 4.4: Detailed use case UC-11: Add or update an analysis model

Field	Description
Actor	Platform Developer / ML Engineer
Preconditions	Trained weights (if any) are available.
Main flow	1. Developer copies the template model folder and edits `manifest.yaml` (key, name, requires, outputs, colour map). 2. Developer implements `run()` in `task.py`: read the image, write a GeoTIFF, return it with class names and metrics. 3. Weights are placed in the model artifact directory. 4. API and worker are restarted; the registry discovers the model and stores it in the catalogue. 5. The model appears on the Models page and in Analyze Imagery; its outputs become layers with legends and statistics.
Alternative flows	4a. Invalid manifest: the error is shown on the Models and Status pages; other models keep working.
Related requirements	FR-REG-01 to FR-REG-05, NFR-R4

FR-ING-06 The system shall store optional location and capture date with an upload.

FR-ING-07 The system shall allow models to be re-run on an existing image.

4.4.2 Module 2: Model Registry and Crop Mapping

FR-REG-01 The system shall discover models from folders containing a manifest and a task.

FR-REG-02 The system shall list registered models with status, inputs, outputs, valid regions and stored accuracy.

FR-REG-03 The system shall resolve model dependencies and run them in order.

FR-REG-04 Placeholder models shall return “not available” and never produce data.

FR-REG-05 A model failure or invalid manifest shall not stop other models from loading.

FR-MAP-01 The system shall classify each pixel as cropland or non-cropland.

FR-MAP-02 The system shall classify the crop type of cropland pixels into the supported classes, grouping rare classes as “Other”.

4.4.3 Module 3: Catalogue, Storage and Statistics

FR-CAT-01 The system shall store areas, images, runs, layers, statistics, field regions and jobs in a spatial database.

FR-CAT-02 The system shall compute area per class for each published categorical layer.

FR-CAT-03 The system shall derive field regions with area from crop-type output and publish them as vector tiles.

FR-CAT-04 Published files shall be immutable and versioned.

4.4.4 Module 4: Web GIS Portal

FR-VIS-01 The portal shall show a home page and navigation to Dashboard, Live Map, Analyze Imagery and Models.

FR-VIS-02 The Live Map shall build its layer panel from the catalogue, grouped by type, with toggles, opacity and legends.

FR-VIS-03 The Live Map shall render raster layers as XYZ tiles and vector layers as PMTiles.

FR-VIS-04 Clicking the map shall return values and labels of all visible layers.

FR-VIS-05 Users shall filter the map by crop and search field regions by crop.

FR-VIS-06 The portal shall switch between areas and between acres and hectares.

FR-VIS-07 The dashboard shall show area by class, shares and model accuracy.

FR-VIS-08 The portal shall show system health and configuration notes.

FR-AUT-01 Users shall be able to create an account and sign in; sign-in can be required by configuration.

4.5 Non-Functional Requirements

4.5.1 Reliability

NFR-R1 The whole stack shall start with one command and recover from start-up ordering (health-checked dependencies, retrying storage initialisation).

NFR-R2 Failed jobs shall record a readable error and leave existing data unchanged.

NFR-R3 Rebuilding one service shall not require restarting the others.

NFR-R4 One broken model shall not prevent other models from loading.

4.5.2 Usability

NFR-U1 Every layer shall have a legend with readable class names.

NFR-U2 Errors shall explain what went wrong and how to fix it.

NFR-U3 The interface shall work at phone and desktop widths without horizontal scrolling.

NFR-U4 Accuracy figures shall be shown with their evaluation protocol.

4.5.3 Performance

NFR-P1 One screen of an AOI shall need under 1 MB of map data.

NFR-P2 Cold tiles shall return in under 300 ms (median); cached tiles in under 50 ms.

NFR-P3 Eight or more layers shall be usable at once without visible lag.

NFR-P4 An AOI-sized image shall be processed by cropland and crop-type models in under 2 minutes on a laptop CPU.

NFR-P5 The selected crop-type model shall run on CPU and be small (target: tens of MB or less).

4.5.4 Security

NFR-S1 Passwords shall be stored hashed (bcrypt); sessions use signed tokens.

NFR-S2 Secrets shall come from environment variables, not code.

NFR-S3 Users shall only see their own and public areas and layers.

NFR-S4 Upload URLs shall be short-lived and limited to one object.

NFR-S5 The tile service shall take styles from the server, not from request parameters.

4.6 Acceptance Criteria

4.7 Requirements Traceability

Table 4.5: Acceptance criteria for important requirements

Requirement	Acceptance check	Result
NFR-P1	Benchmark: data for one screen of AOI1 at zoom 12 below 1 MB.	337 KB, pass
NFR-P2	Benchmark: cold tile median below 300 ms; cached below 50 ms.	47 ms / 1 ms, pass
NFR-P4	Upload AOI1 Sentinel-2 image, run both models, layers listed.	57 s, pass
FR-REG-01	Copy the template model, edit only the manifest; model loads and runs.	Pass (TC-03)
FR-REG-04	Run yield placeholder; no files produced.	Pass (TC-04)
FR-ING-03	Upload a 1-band file; job fails with a readable reason.	Pass (TC-13)
FR-VIS-04	Click a pivot field; crop type and cropland labels returned.	Pass (TC-08)
FR-VIS-05	Filter “Apples”; only apple pixels remain on the map.	Pass (TC-09)
NFR-R1	`dockercomposedown` then `up-d`; health OK for all services.	Pass (TC-15)
FR-MAP-02	Crop-type accuracy and macro-F1 on spatial test blocks reported with intervals.	76.1% / 92.2%, reported

Table 4.6: Traceability of key requirements to modules, design, implementation and tests

Req.	Module	Design element	Implementation	Test
FR-ING-02	M5	Signed upload (Fig. 5.3)	`routes.py:create_upload`, `storage.py`	TC-11
FR-ING-03/04	M5, M1	Ingest task	`tasks.py:ingest_upload`, `rasters.py`	TC-11, TC-13
FR-ING-05	M5	Job states (Fig. 5.4)	`tasks.py:run_pipeline`, `Job`	TC-11, TC-12
FR-REG-01/03	M6	Registry, graph	`registry.py`, `graph.py`	TC-01, TC-02, TC-03
FR-REG-04	M6	StubTask	`base.py:StubTask`	TC-04
FR-MAP-01/02	M2	Model tasks	`builtin/cropland`, `builtin/croptype`	TC-11, Ch. 6
FR-CAT-02	M7	ZonalStat	`rasters.py:categorical_stats`	TC-07
FR-CAT-03	M3	PMTiles layer	`vector_tiles.py`	TC-10
FR-VIS-03	M8	Tile service	`tiles/main.py`, `LiveMap.tsx`	TC-05, TC-06
FR-VIS-04	M8	Point query	`routes.py:point`	TC-08
FR-VIS-05	M8	Crop filter	`tiles/main.py:classes`, `LiveMap.tsx`	TC-09
NFR-P1/P2	M8	Tiles, cache	`nginx.conf`, `benchmark_tiles.py`	TC-14
NFR-R1	–	Compose health checks	`docker-compose.yml`, `minio-init.sh`	TC-15

Chapter 5

System Design

5.1 System Architecture

GeoCrop follows a layered, service-oriented architecture built around one principle: *send only what is visible*. Model results are stored once as Cloud Optimized GeoTIFFs (COG) in object storage; the browser requests small map tiles for the current view, which a stateless tile service cuts from the COG and Nginx caches. Heavy work runs in background workers, and a spatial database holds the catalogue that tells the interface what exists. The main building blocks are the COG format [21], TiTiler [22], PMTiles [23], MapLibre GL [24] and PostGIS [25]. Figure 5.1 shows the services and their connections.

5.2 Architectural Components

5.3 Interfaces and Dependencies

External dependencies are Sentinel-2 imagery exported with Google Earth Engine, USDA CDL rasters and AOI shapefiles, an OpenStreetMap or MapTiler basemap, and the open-source libraries listed in Section 8.8. Internally, the API depends on the database, broker and storage; the tile service depends on the API (layer lookup, cached for five minutes) and storage; workers depend on the broker, database, storage and model artifacts.

5.4 Architecture Decisions and Rationale

5.5 Design Evolution

The proposal-stage prototype was a Django application with Leaflet. It returned the crop-type result for AOI1 as one GeoJSON response; the browser parsed it and drew 101,435 polygons as SVG. Uploads were recorded but never processed, field data on the dashboard was hand-

Table 5.1: Architectural components

Component	Technology	Responsibility
Web GIS client	React, Vite, TypeScript, MapLibre GL via react-map-gl, TanStack Query, Zustand	Home, Live Map workspace, Dashboard, Analyze Imagery, Processing, Models, Status, accounts. Builds layers from the catalogue; loads tiles on demand.
Edge	Nginx	Serves the built app; proxies the API; caches tiles with immutable headers; resolves services through Docker DNS at request time.
API service	FastAPI, Pydantic, SQLAlchemy 2, GeoAlchemy2, PyJWT, bcrypt	REST API v1: health, auth, areas, models, layers, fields, point query, statistics, uploads, runs, jobs.
Tile service	TiTiler / rio-tiler (FastAPI)	`/tiles/{layer}/{version}/{z}/{x}/{y}.png`; reads only needed COG blocks; applies the server-side style; optional crop filter.
Workers	Celery	Ingest (validate, reproject, COG), model runs through the registry, statistics, layer publishing, progress.
Model registry	Python package with YAML manifests	Discovers models, resolves dependencies, runs tasks with a common context; built-in cropland, crop type, NDVI index, boundary and yield placeholders.
Spatial database	PostgreSQL 16 + PostGIS 3.4, Alembic	Catalogue of users, areas, uploads, images, models, runs, layers, statistics, field regions, jobs.
Job broker	Redis 7	Celery broker and result backend.
Object storage	MinIO (S3 API)	Uploads, COG imagery and results, PMTiles; anonymous read only for vector tiles.

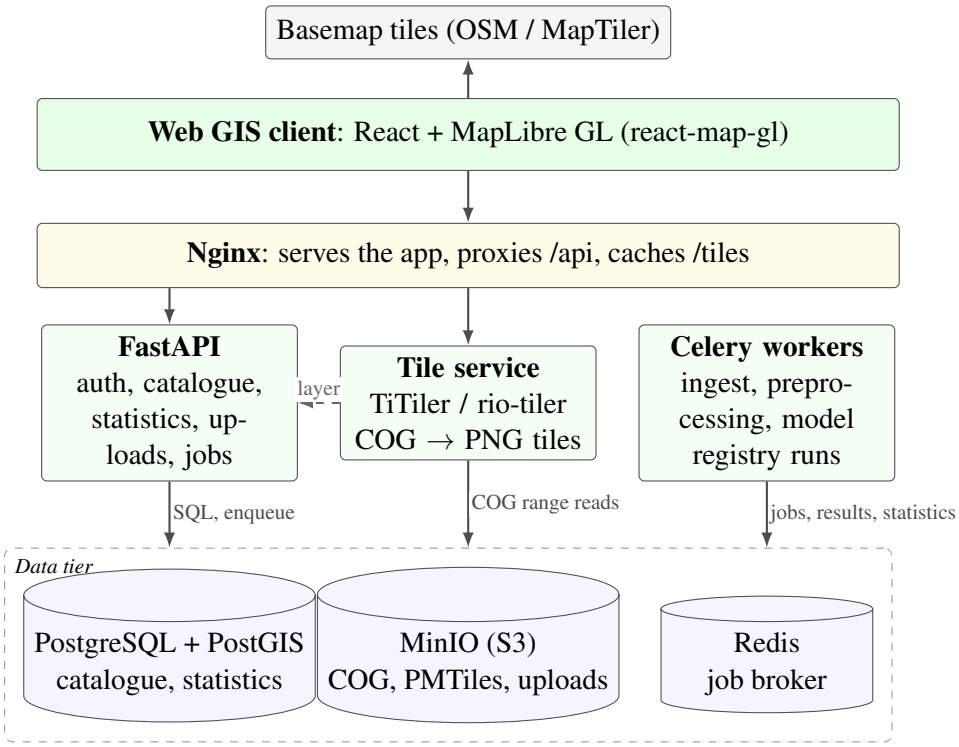


Figure 5.1: GeoCrop system architecture. The browser also reads PMTiles and uploads imagery directly to object storage using short-lived signed URLs; the tile service looks up layer styles from the API.

Table 5.2: Main REST interfaces (prefix `/api/v1`; interactive documentation at `/api/docs`)

Endpoint	Purpose
`GET/health`	Status of database, broker, storage, tiles and model registry; configuration notes.
`POST/auth/signup`, `/auth/login`, `GET/auth/me`	Accounts and tokens.
`GET/aois`	Areas visible to the user, with bounds.
`GET/models`	Registered models, status, inputs, outputs, accuracy and per-class scores.
`GET/layers?aoi=&kind=&run=`	Layer catalogue: style, legend, bounds, zoom range, tile URL, version.
`GET/fields?aoi=&q=`	Field regions with crop, area and bounding box.
`GET/point?lng=&lat=&layers=`	Values and labels of several layers at one location.
`GET/stats?layer=`	Pre-computed area and share per class.
`POST/uploads`, `/uploads/{id}/complete`	Signed upload URL; start ingestion with models, location and date.
`POST/runs`, `GET/runs`, `GET/jobs/{id}`	Re-run models; run history; job progress and steps.
`GET/tiles/{layer}/{v}/{z}/{x}/{y}.png`	Raster tiles (tile service, cached by Nginx).

Table 5.3: Key architecture decisions

Decision	Rationale	Alternative rejected
Raster as source of truth (COG)	Models predict pixels; a 30 m raster of AOI1 is about 1.6 MB, the same result as polygons 43.3 MB.	Vectorising all output to GeoJSON
Tiles via TiTiler	Reads only the COG blocks needed per tile; styles applied on the server; no pre-rendering.	Pre-rendered tile folders; single PNG overlays
PMTiles for vectors	One static file read by range requests; no tile server; cacheable.	GeoJSON; dynamic vector tile server
MapLibre GL	WebGL rendering of raster and vector tiles; open source.	Leaflet (SVG/Canvas)
Versioned, immutable tile URLs	Browsers and Nginx can cache for a year; a new run gets a new URL.	Cache invalidation logic
FastAPI	Same stack as TiTiler; async I/O; typed schemas and generated docs.	Django templates (prototype)
PostgreSQL + PostGIS	Spatial types and indexes, joins between catalogue and statistics, JSONB for styles and metrics.	SQLite (prototype)
Celery + Redis	Long jobs off the request path; retries; progress.	Synchronous processing
S3-compatible storage (MinIO)	Large files, range reads, signed uploads, cloud-portable.	Local disk
Manifest-based model registry	Models change over the project; new models without changes elsewhere.	Hard-coded model calls
Docker Compose	One-command, reproducible stack for the team and the panel.	Manual installation

written, and the session cookie held upload history. Table 5.4 summarises the changes and why they were made.

Table 5.4: Design evolution since the proposal

Aspect	Proposal prototype	Current design	Reason
Result format	GeoJSON polygons (43.3 MB)	COG rasters + PMTiles	Data volume and rendering cost
Map client	Leaflet, SVG	MapLibre GL, WebGL tiles	Smooth zoom with many layers
Backend	Django views and templates	FastAPI + React SPA	Async API, typed contracts, separation of concerns
Database	SQLite	PostgreSQL + PostGIS	Spatial queries, concurrency, catalogue
Processing	None (upload recorded only)	Celery jobs with progress	Real analysis of new imagery
Models	Offline scripts writing GeoJSON	Registry plug-ins writing rasters	Add models without code changes
Dashboard data	Hard-coded sample fields	Pre-computed statistics in PostGIS	Real results
Deployment	Single process	Seven containers via Compose	Reproducible, scalable services

5.6 Domain Model

Figure 5.2 shows the main domain classes. An *Area of Interest* owns source images and model runs; a run executes one registered *Model* and produces one or more *Layers*; categorical layers are summarised by *Zonal statistics*; crop-type runs also produce *Field regions*. A *Job* tracks the steps of an upload or a run.

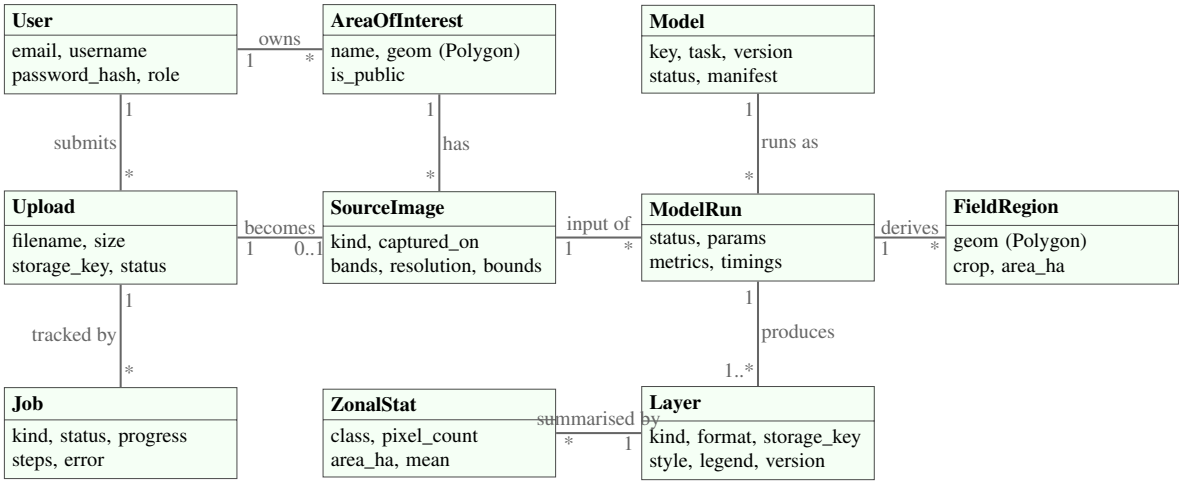


Figure 5.2: Domain model

5.7 Data Design

Relational schema. The catalogue is a PostgreSQL/PostGIS schema managed with Alembic (Table 5.5). Geometries are stored in EPSG:4326 with GiST spatial indexes. Free-form fields (layer kind, style, legend, model manifest, run metrics) are strings or JSONB, so a new model output never needs a schema change. Rasters are never stored in the database.

Table 5.5: Database tables

Table	Key columns
`users`	id, email (unique), username (unique), password_hash, role, created_at
`aoi`	id, owner_id, name, geom (Polygon, GiST), is_public
`upload`	id, user_id, aoi_id, filename, size_bytes, storage_key, status
`source_image`	id, aoi_id, upload_id, kind, captured_on, storage_key, bounds (GiST), band_count, resolution_m
`model`	id, key (unique), task, version, name, status, manifest (JSONB), enabled
`model_run`	id, aoi_id, model_id, source_image_id, status, params, metrics (JSONB), error, timings
`map_layer`	id, run_id, aoi_id, kind, group, name, format, storage_key, bounds (GiST), zoom range, nodata, style, legend (JSONB), version, is_public
`zonal_stat`	layer_id, zone_type, zone_id, class_code, class_name, pixel_count, area_ha, mean, min, max
`field`	id, aoi_id, run_id, geom (Polygon, GiST), attributes (JSONB)
`job`	id, kind, status, progress, message, error, steps (JSONB), upload_id, user_id

Object storage layout (bucket `geocrop`): `raw/{upload}/original.tif` for uploads; `runs/reference/...cog.tif` for imagery and reference layers; `runs/{run}/{kind}-{id}.cog.tif` for model outputs; and `vector/{aoi}/{run}/...pmtiles` for vector layers. Files are written once; a new run creates new keys.

Raster conventions. COGs use 512-pixel internal blocks, DEFLATE compression and overviews (nearest neighbour for class maps, average for imagery). Categorical rasters use 0 as no-data; the cropland map uses 1 = cropland and 2 = non-cropland; crop-type codes map to class names stored in the layer legend.

5.8 Process / Behavior Design

Upload to map. Figure 5.3 shows the interaction from upload to published layers.

Job and run states. A job moves from *queued* to *running* and ends in *done* or *failed*; each step inside it is *pending*, *running*, *done*, *skipped* (placeholder model) or *failed* (Figure 5.4).

Model execution. The registry plans the requested models with a topological sort over the layer kinds they require and produce (for example, crop type requires cropland). Each model

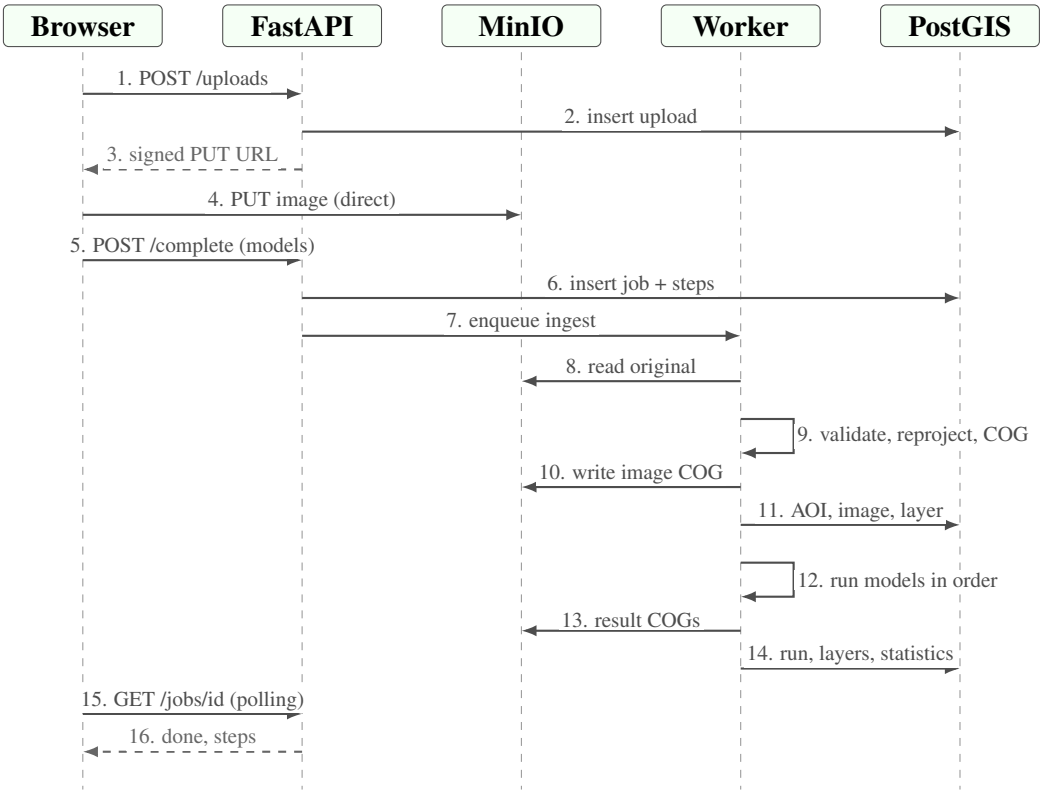


Figure 5.3: Sequence of an upload and analysis

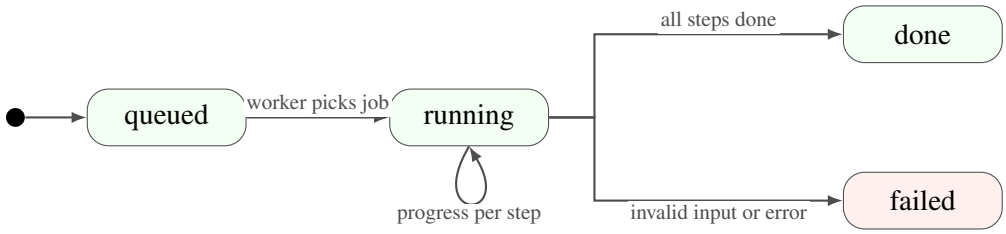


Figure 5.4: Job state machine

receives a run context (image path, upstream outputs, scratch folder, artifacts, progress callback); its output files are converted to COG, uploaded, registered as layers with style and legend, and summarised as zonal statistics before the next model starts, so the map fills in progressively.

Tile request. The client requests `/tiles/{layer}/{version}/{z}/{x}/{y}.png`. Nginx returns a cached copy when it has one; otherwise the tile service looks up the layer (cached), reads the needed COG window from storage with nearest-neighbour resampling for class maps, applies the stored colour map and returns a PNG with a one-year immutable cache header. Tiles outside the raster return HTTP 204.

Chapter 6

Iteration 1: Design and Implementation

6.1 Iteration Objectives

1. **Architecture track:** design and implement a base architecture that keeps the map fast as images, models and layers grow, and prove it on the real AOI1 data.
2. **Architecture track:** turn the prototype's "upload and nothing happens" into a real pipeline in which an uploaded image is analysed and published automatically.
3. **Architecture track:** make models replaceable, so research results can be integrated without redesign.
4. **Model track:** evaluate crop-type models under a fair spatial test, measure their cost, improve the best candidates, and compare with published work.

6.2 Selected Use Cases / Features

6.3 Design

The architecture track implements the design of Chapter 5: Figure 5.1 (services), Figure 5.3 (processing), Figure 5.2 and Table 5.5 (data). The model track follows the experiment design in Figure 6.1: every experiment uses the same inputs, split, classes and seed, and only the validation set decides.

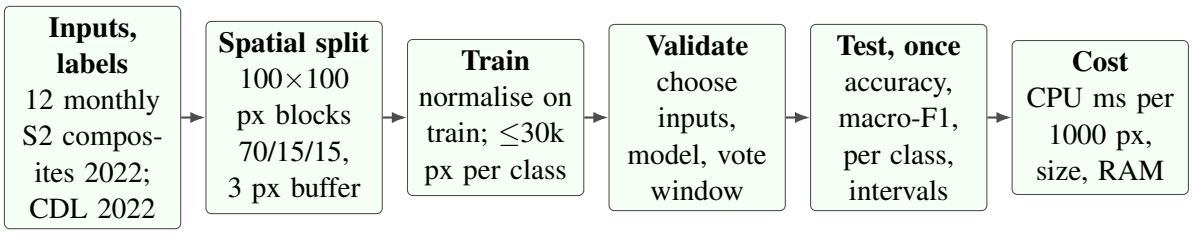


Figure 6.1: Crop-type experiment design

Table 6.1: Use cases and features selected for Iteration 1

Use case	Feature	Track
UC-01, UC-02	Tile-based Live Map with layer panel, legends, opacity, point inspection	Architecture
UC-03	Crop filter, crop ranking and searchable field list	Architecture
UC-04	Dashboard with pre-computed statistics and model accuracy	Architecture
UC-05, UC-06, UC-12	Upload, background processing, live progress	Architecture
UC-07	Re-analyse an existing image	Architecture
UC-08, UC-11	Model catalogue and plug-in registry	Architecture
UC-09, UC-10	Accounts and system status	Architecture
FR-MAP-02	Crop-type model comparison, selection and error analysis	Model

6.4 Implementation

Architecture track. The platform was implemented as a new repository (`geocrop-platform`) and runs as seven containers with Docker Compose: `web` (Nginx with the built React app), `api` (FastAPI), `worker` (Celery), `tiles` (TiTiler), `postgres` (PostGIS), `redis` and `minio`, plus a one-shot bucket initialiser. Table 6.2 gives the size of each part. Data preparation scripts convert the AOI rasters to COG, build the field-region PMTiles and load the AOI1 demonstration data.

Table 6.2: Platform code by component (lines, including configuration and documentation)

Component	Contents	Lines
`web/src`	React pages, map workspace, components, styles	1,998
`backend/app`	API routes, models, services, workers, seed	1,547
`backend/model_registry`	Contract, registry, graph, colour maps, models	498
`ml`	Extended training scripts	373
`docs`	Architecture, status, upgrade path, guides	282
`tools`	COG conversion, boundaries, benchmark	222
`tiles`	Tile service	77
`infra`	Nginx and storage initialisation	68

Model track. Two notebooks implement the experiments: a benchmark notebook (runs 1 and 2, model families, cost measurement, spatial bootstrap) and an accuracy-improvement notebook (run 3, for AOI1 or AOI2). Inputs are 12 monthly Sentinel-2 composites of 2022

with 11 values per month (B2, B3, B4, B5, B6, B7, B8, B8A, B11, B12 and NDVI) and the CDL 2022 rasters, all on the same 30 m EPSG:5070 grid. AOI1 (eastern Washington, about 40×37 km, 14 classes, 99,238 test pixels) is a hard area with small fragmented fields, orchards and similar hay crops; AOI2 (north-eastern North Dakota, about 38×29 km, 9 classes, 144,922 test pixels) has large regular fields and two dominant crops.

6.5 Completed Features / Module

Table 6.3: Completed features at mid-term

Module	Completed	State
M1 Data preparation	COG conversion with validation; AOI1 Sentinel-2 composite and CDL as COG; monthly 2022 composites for AOI1 and AOI2	Done
M2 Crop mapping	Platform: cropland and crop-type Random Forests trained and run on uploads. Research: 11 models compared; improved LightGBM pipeline selected	Done
M3 Field regions	2,433 field regions (at least 25 pixels) as a 1.86 MB PMTiles layer and PostGIS rows	Interim
M4 Yield	Registered placeholder; NASS download script prepared	Planned
M5 Processing pipeline	Signed uploads, validation, reprojection, COG, background runs, progress, re-runs	Done
M6 Model registry	Contract, manifests, dependency graph, placeholders, NDVI template, add-a-model test	Done
M7 Catalogue and statistics	PostGIS schema with Alembic migration; zonal statistics; field search	Done
M8 Web GIS portal	Home, Live Map workspace, Dashboard, Analyze Imagery, Processing, Models, Status, accounts	Done

6.6 Technical Implementation Details

Configuration is read from environment variables (`.env`); the database schema is created by an Alembic migration that manages only GeoCrop's tables (the PostGIS image ships its own topology and geocoder tables, which are excluded).

6.7 Algorithms and Technical Decisions

6.7.1 Platform algorithms

- **COG conversion:** GDAL COG driver, 512-pixel blocks, DEFLATE, overviews with nearest neighbour for categorical rasters and average for imagery; validated by layout, tiling, overviews and CRS.

Table 6.4: Important files, classes and functions

File / element	Role
`model_registry/base.py`: `ModelTask`, `RunContext`, `RunResult`, `StubTask`	Contract every model implements; placeholders never fabricate data.
`model_registry/registry.py`: `Registry`	Scans folders for `manifest.yaml`, imports the entry-point, isolates broken models.
`model_registry/graph.py`: `execution_order`	Topological order from required and produced layer kinds; readable errors for cycles and missing producers.
`builtin/cropland`, `builtin/croptype`	Windowed (512-pixel block) inference with saved models; crop type runs only on cropland pixels.
`app/workers/tasks.py`: `ingest_upload`, `run_pipeline`	Validate, reproject, convert, publish image; run models; publish layers and statistics; update job steps.
`app/services/layers.py`: `publish_raster_layer`	COG conversion, upload, footprint, layer row and zonal statistics in one step.
`app/services/rasters.py`	Footprint, point sampling, block-wise class statistics, reprojection to the model grid.
`app/services/vector_tiles.py`	Polygonise class rasters, simplify per zoom, encode MVT, write PMTiles.
`app/services/storage.py`	One storage interface for S3/MinIO and local disk; GDAL `/vsis3/` access.
`tiles/main.py`	Tile endpoint with server-side styles, crop filter, immutable caching.
`web/src/pages/LiveMap.tsx`	Map workspace; layers from the catalogue; filter, field search, inspection, tile traffic panel.
`docker-compose.yml`, `web/nginx.conf`	Services, health checks, tile cache, runtime DNS resolution.

- **Tile rendering:** for tile (z, x, y) the service reads the overlapping COG window at the matching overview, resamples (nearest for classes), maps class codes to the stored colours (optionally only selected classes) and encodes a PNG.
- **Windowed inference:** images are processed in 512×512 windows; features follow the training composite (B2, B3, B4, B8, B11, B12, NDVI; NDVI computed when absent).
- **Dependency resolution:** depth-first topological sort over model manifests.
- **Zonal statistics:** block-wise class histograms; area = pixels × pixel area (900 m^2 = 0.09 ha at 30 m).
- **Field regions and PMTiles:** connected same-class regions of at least 25 pixels, simplified, encoded as MVT for zooms 8–14 (507 tiles) and packed into one PMTiles file.

6.7.2 Crop-type evaluation protocol

- **Spatial block split:** whole 100×100-pixel blocks go to training (70%), validation (15%) or test (15%), with a 3-pixel buffer between them. A random pixel split leaks neighbours: on the same model it gave macro-F1 0.396 against 0.147 with blocks.
- Normalisation fitted on training pixels only; validation macro-F1 decides inputs, model and vote window; test results are computed once.
- Classes under 5,000 pixels are merged into "Other (rare classes)"; up to 30,000 training pixels per class with balanced class weights; same split, classes and seed for every model.
- Cost is measured as CPU milliseconds per 1000 pixels, model size and RAM in a fresh single-thread process, as requested by the panel.
- Uncertainty: paired block bootstrap over the test blocks (4,000 resamples).

6.7.3 Results: three runs on AOI1

Table 6.5: Progress over three runs on AOI1 (all test pixels)

Run	Input	Model	Accuracy	Macro-F1
1	One composite (single date)	Random Forest (3 seeds)	0.267	0.191
2	12 monthly composites of 2022	Random Forest (3 seeds)	0.752	0.687
3	+ neighbourhood inputs + neighbour vote	Average of 3 models	0.761	0.686

Run 1 to 2 changed two things together: a time series instead of one date, and a satellite year that matches the CDL year, so their separate effects are not known.

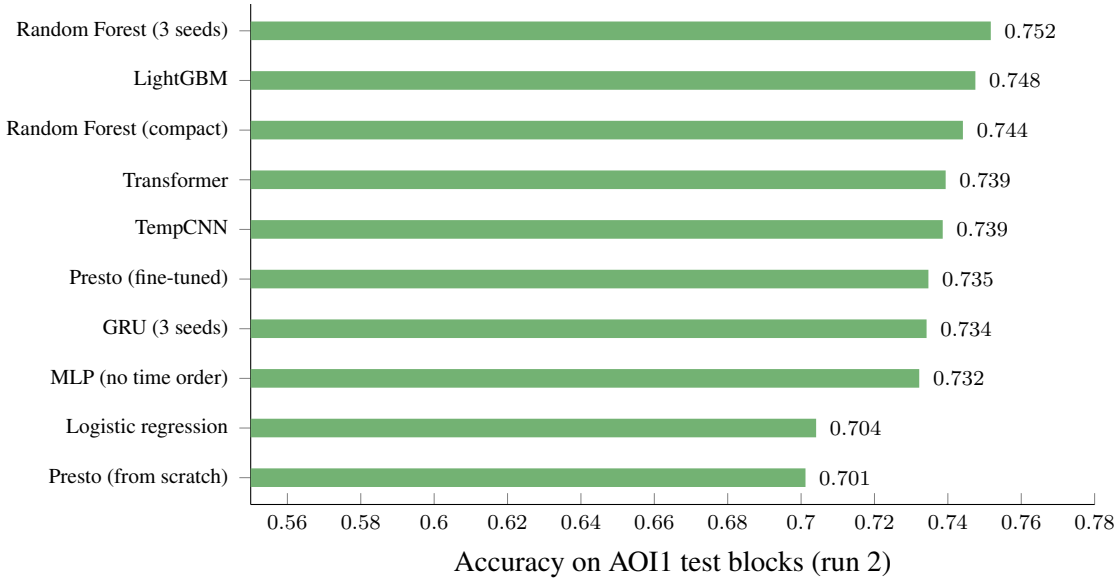


Figure 6.2: Run 2: eleven models on the same AOI1 test blocks (axis starts at 0.55; majority class 0.199 not shown)

Eight model families end within 2 points (73.2% to 75.2%); logistic regression is 4.8 points behind; models that use time order (GRU, TempCNN [18], Transformer) do not clearly beat an MLP that ignores it. Data and labels set the ceiling, not the algorithm, so the choice was made on cost (Table 6.6); LightGBM [19] was proposed.

Table 6.6: Accuracy against cost (run 2, AOI1; single-thread CPU)

Model	Accuracy	CPU ms / 1000 px	Size (MB)	RAM (MB)
Logistic regression	0.704	0.016	1.2	321
MLP (no time order)	0.732	3.8	0.29	550
Random Forest (compact)	0.744	13.5	57.4	397
Random Forest (3 seeds)	0.752	24.3	327.8	809
TempCNN	0.739	31.9	0.6	570
LightGBM (proposed)	**0.748**	**35.8**	**1.98**	**549**
GRU (3 seeds)	0.734	70.0	0.43	600
Transformer	0.739	79.9	0.29	588
Presto (fine-tuned)	0.735	2551	1.65	1200

6.7.4 Run 3: improvement ladder

An improvement was kept only if validation macro-F1 rose by at least 0.002. Neighbourhood inputs are the mean of NDVI, B8 and B11 over 3×3 and 7×7 windows and the spread of NDVI; the 7×7 window reaches 3 pixels, equal to the buffer, so no training pixel overlaps a test block. Cleaning labels (removing edge pixels from training) hurt, because 29–47%

Table 6.7: Improvement ladder (inputs per month; AOI1 validation change; AOI2 scores)

Step	Inputs / month	AOI1 val. change	AOI2 val. macro-F1	AOI2 test accuracy	Decision
0 Benchmark (raw inputs)	11	start	0.731	0.903	start
1 + 4 indices, month flag	16	+0.0017	0.695	0.889	dropped
2 + cleaned training labels	11	−0.030	0.681	0.909	dropped
3 + neighbourhood inputs	19	+0.0024	0.742	0.912	kept

of test pixels are edge pixels. The 3×3 neighbour vote averages predicted probabilities of nearby test pixels; window 3 was best on validation in both areas.

6.7.5 Run 3 results in both areas

Table 6.8: Final results on both areas (all test pixels)

	AOI1 (E. Washington)	AOI2 (NE North Dakota)
Classes / test pixels	14 / 99,238	9 / 144,922
Benchmark: accuracy / macro-F1	0.741 / 0.661	0.903 / 0.759
Improved inputs	0.756 / 0.685	0.912 / 0.780
Improved + neighbour vote	**0.761 / 0.686**	**0.922 / 0.803**
Gain over benchmark (points)	+2.0 / +2.5	+1.8 / +4.4
Test pixels on field edges	47.0%	29.4%
Accuracy on interior / edge pixels	0.952 / 0.546	0.988 / 0.762
Share of errors on edge pixels	89%	89%
Final model (chosen on validation)	Average of 3 models	LightGBM

How to quote the result. AOI1: 76.1% on all test pixels (95% interval 72.1% to 79.6% over test blocks), macro-F1 0.686; on field interiors 95.2%. The improved pipeline is +2.0 points over the benchmark (interval +1.0 to +3.0). Run-to-run noise is about 0.7 points, so differences under about 1 point are not reliable.

Error analysis. About 89% of errors in both areas sit on pixels that touch another label; 67% of AOI1 edge errors predict the neighbouring class (boundary bleed). Confusions are between agronomically close classes: Alfalfa and Other Hay, Corn and Sweet Corn (AOI1); Spring Wheat and Barley, Spring Wheat and Fallow/Idle (AOI2). Weakest AOI1 classes by F1 are Fallow/Idle (0.29) and Other (0.29); strongest are Onions (0.91), Potatoes (0.86) and Alfalfa (0.85) (Appendix C, Table C.2).

6.7.6 Comparison with published work

AOI2 (92.2%) is close to the 94% reported for South Dakota, a similar northern-plains crop mix, while using a stricter spatial split and no label-quality filter. AOI1 (76.1%) is below the

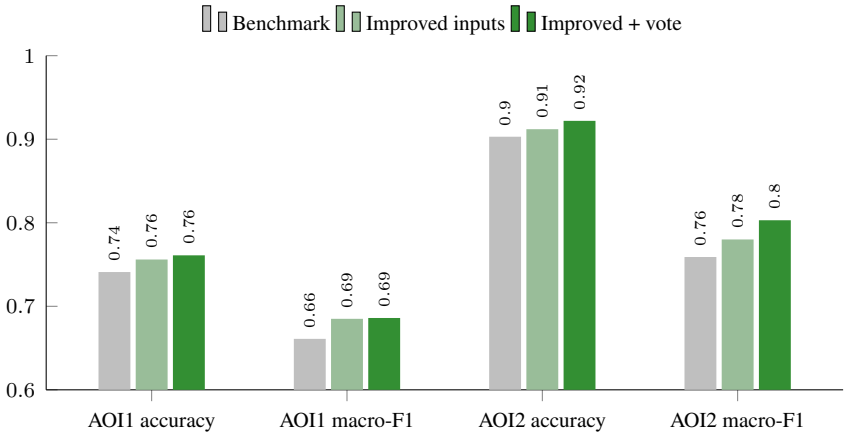


Figure 6.3: Benchmark, improved inputs and neighbour vote in both areas (axis starts at 0.6)

Table 6.9: GeoCrop crop-type results next to published studies (context, not a ranking)

Study	Unit	Labels / split	Result	Classes
GeoCrop (ours)	Pixel, 30 m	CDL 2022, all pixels; spatial blocks	AOI2 92.2%; AOI1 76.1% (interior 95.2%)	9 and 14
Tran et al. 2022 [3]	Pixel, 10 m	Good-quality CDL; 80/20 samples	94% South Dakota; 83% California	10 and 20
Li et al. 2024 [13]	Pixel	Trusted CDL pixels	Corn/soybean F1 0.85–0.97	–
PSE-TAE 2020 [15]	Parcel	Parcel declarations	OA 94.2%; RF 91.6%	20
BreizhCrops 2020 [17]	Parcel	Held-out region	Transformer 0.81, TempCNN 0.80, RF 0.78	9
Presto 2023 [16]	Pixel series	Pre-trained	S2-Agri100 OA 68.9%	–
Prithvi-EO-2.0 2024 [14]	Chip	CDL, 3 dates	mIoU 48.6–50.7	13

83% for California but is a harder area with 47% edge pixels. Parcel-level studies classify whole fields, which removes the edge problem, so their numbers are not directly comparable. As in the papers, the algorithm moves results by a few points, while labels and the evaluation unit move them by tens of points. A like-for-like comparison needs the CDL confidence filter and matching class sets (Iteration 2).

Baseline model in the platform. The platform currently runs the proposal-stage two-stage Random Forest (single composite, seven features), retrained and saved for live inference. Its held-out scores of 93% (cropland) and 56% (crop type) come from a random pixel split and are shown in the portal with that caveat; under the spatial protocol a single-date input reaches only 26.7%. Replacing it with the selected pipeline is the first task of Iteration 2.

6.8 Tools, Frameworks and Technologies

Table 6.10: Tools, frameworks and technologies

Area	Tools
Frontend	React 19, Vite, TypeScript, MapLibre GL JS, react-map-gl, pmtiles, TanStack Query, Zustand, React Router
Backend	Python 3.12, FastAPI, Pydantic, SQLAlchemy 2, GeoAlchemy2, Alembic, PyJWT, bcrypt, boto3
Geospatial	GDAL/rasterio, rio-tiler, TiTiler, shapely, pyproj, mercantile, mapbox-vector-tile, PostGIS 3.4
Processing	Celery, Redis 7
Storage	MinIO (S3 API), PostgreSQL 16
Machine learning	scikit-learn, LightGBM, PyTorch (deep models in the comparison), Presto, joblib, NumPy
Data	Google Earth Engine (Sentinel-2 export), USDA CDL, USDA NASS QuickStats
DevOps	Docker, Docker Compose, Nginx, Git
Testing	pytest, curl-based API checks, headless browser checks, custom tile benchmark

6.9 Demonstration Evidence

Rendering benchmark. Table 6.11 compares the prototype's approach with the new tile path on the same AOI1 crop-type result, measured through Nginx on a laptop.

Pipeline run. Uploading the AOI1 Sentinel-2 GeoTIFF through a signed URL, ingesting it, running the cropland and crop-type models block by block and publishing both layers took 57 s, with each step visible on the Processing page. Loading the AOI1 demonstration data took about 5 s.

Screenshots. Figures 6.5–6.8 show the running portal.

Table 6.11: Rendering benchmark on AOI1 (crop-type layer, zoom 12)

Measure	Prototype (GeoJSON)	GeoCrop (COG tiles)
Data for one screen of AOI1	43.3 MB (whole file)	337 KB (24 tiles)
Objects the browser draws	101,435 polygons (≈750k vertices)	24 PNG images
Server / parse time	1,021 ms to parse the JSON only	cold tile median 47 ms, p95 54 ms
Cached tile latency	no caching	median 1 ms (Nginx hit)
Typical tile size	n/a	median 2.9 KB, max 44 KB
Storage of the same result	43.3 MB GeoJSON	222 KB compressed GeoTIFF

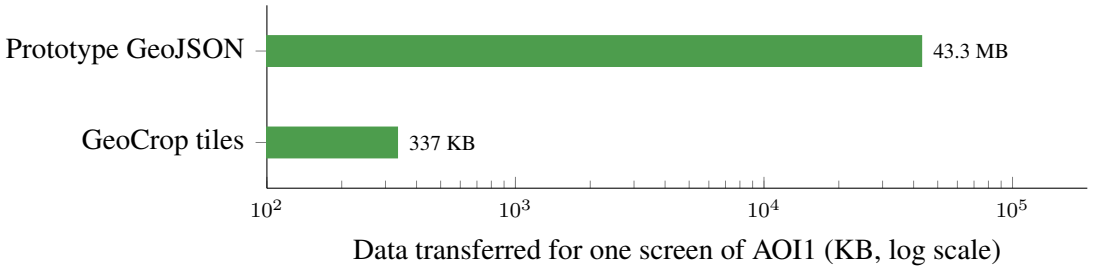


Figure 6.4: Data per screen: prototype against tile-based delivery (about 130 times less)

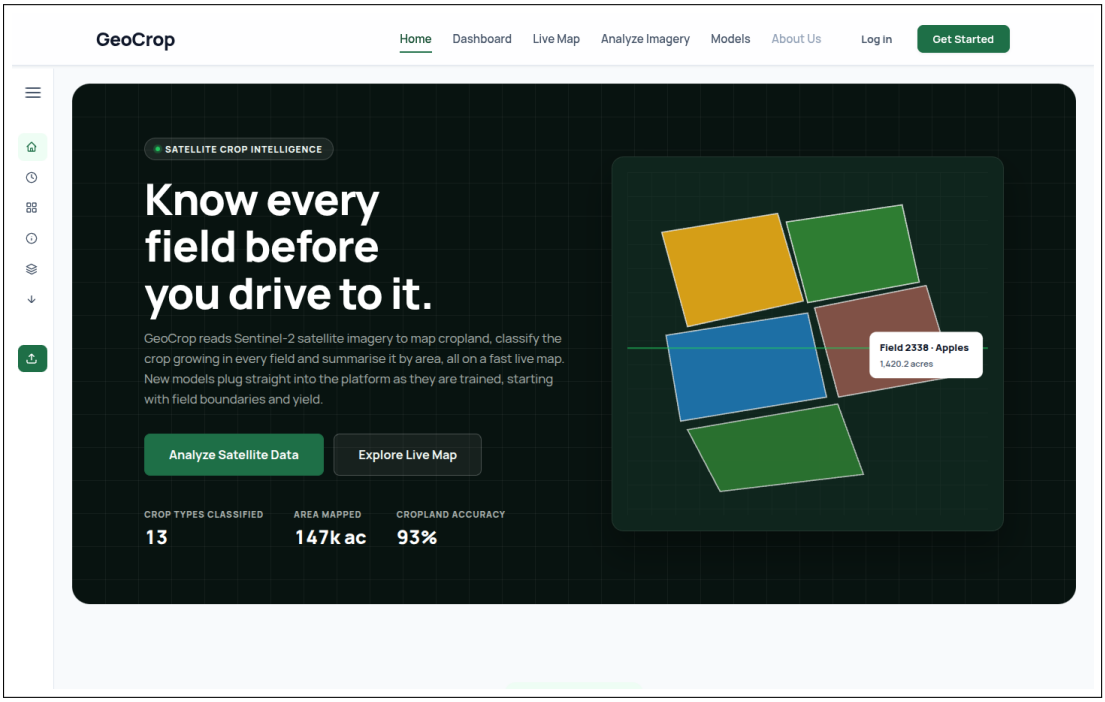


Figure 6.5: Home page with live figures from the AOI1 results

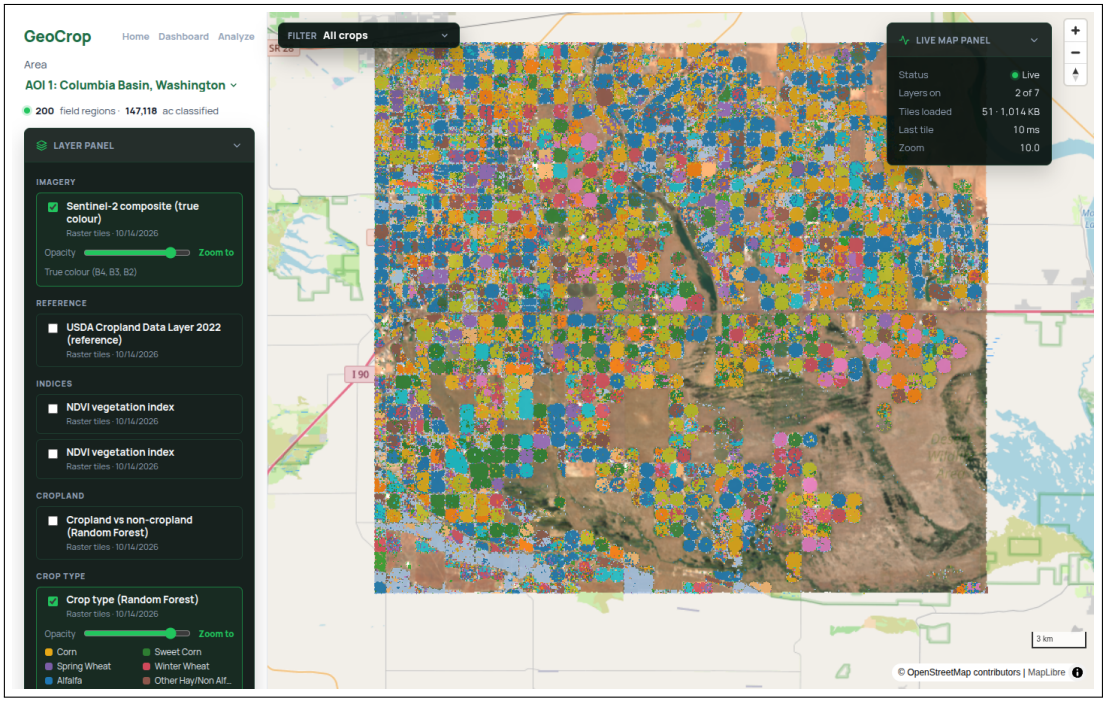


Figure 6.6: Live Map workspace: layer panel, crop-type layer over Sentinel-2 imagery, crop filter and Live map panel (tiles loaded and latency)

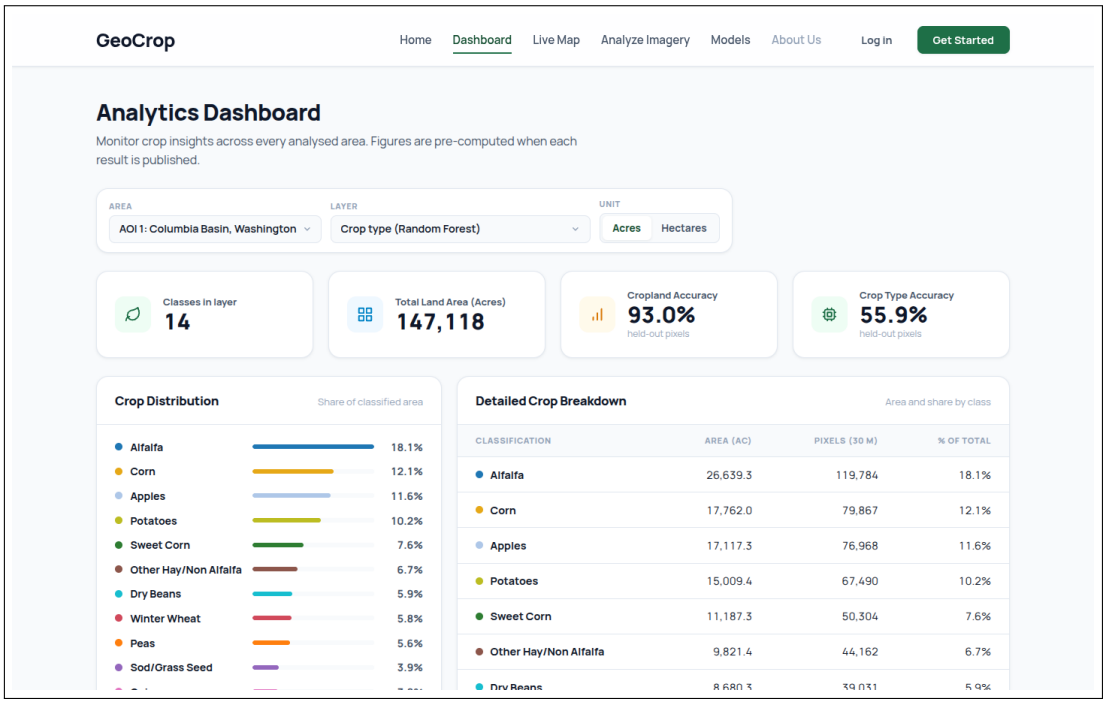


Figure 6.7: Analytics Dashboard: statistics from PostGIS and measured model accuracy

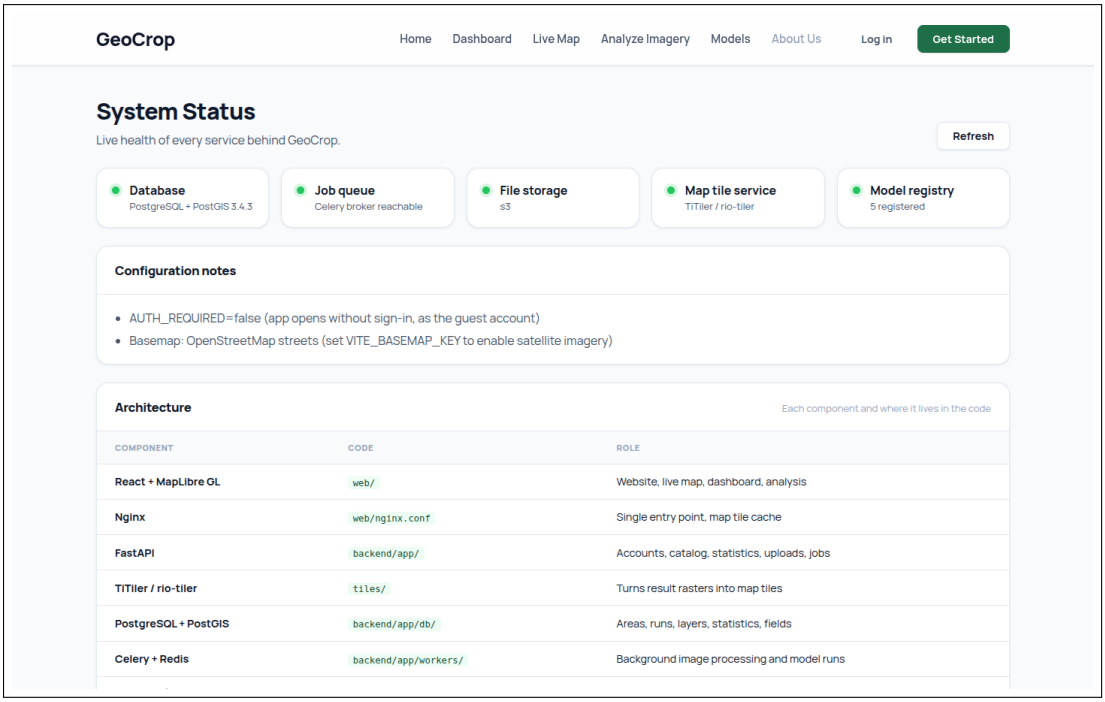


Figure 6.8: System Status: health of every service and the architecture map

6.9.1 Workflow feasibility and resource requirements

The panel found that the complexity and resource requirements of the complete workflow had not been analysed, and that running it as one single flow may not be practical (P4, P5). Each stage was therefore measured on its own on the AOI1 image (1,332×1,239 pixels at 30 m, about 148,500 ha, 7 inputs per pixel). Every stage ran in a fresh process in the worker container, on an 8-core laptop CPU with 15 GB RAM, reading the image in 512×512 blocks (Table 6.12).

Three findings follow. First, the compute itself is small: all model and conversion stages together take about 15 s; the rest of the 57 s run is transfer, queueing and publishing. Second, memory, not time, is the real limit. The baseline Random Forest crop-type model needs 1.57 GB on disk and about 3.1 GB of RAM, so two parallel runs would already need more than 6 GB. The proposed LightGBM model needs 549 MB (measured by the model track, Table 6.6) at about the same accuracy, which makes the full workflow fit on a normal laptop. Third, results are small once stored as compressed rasters (285 KB for both layers against 43.3 MB of GeoJSON), so storage grows slowly as AOIs and models are added.

The design therefore avoids a single flow. Ingest, each model and publishing are separate queued steps that run in background workers, one model at a time in dependency order, with progress and errors recorded per step. A failed or heavy step does not block the map, the API or other jobs, and workers can be added on another machine without changing the code. Users never load the whole dataset: the map requests only the tiles in view.

Table 6.12: Measured resource requirements of the complete workflow (AOI1, one fresh process per stage)

Stage	Time	Peak RAM	Storage / notes
Upload and validation	–	–	Input GeoTIFF 24.8 MB, sent directly to object storage
Cloud-optimised GeoTIFF	0.7 s	357 MB	29.5 MB COG with overviews
Cropland model (RF)	1.1 s load + 2.7 s	395 MB	Model 71.6 MB; 1.11 M pixels classified
Crop-type model (baseline RF)	2.5 s load + 7.5 s	**3,145 MB**	Model **1.57 GB**; 0.69 M cropland pixels
Crop-type model (LightGBM, proposed)	35.8 ms / 1000 px	549 MB	Model 1.98 MB (Table 6.6)
Publish results	–	–	Crop type 222 KB, cropland 63 KB compressed rasters; statistics in PostGIS
Full upload-to-map run	57 s	–	Includes transfer, queueing, storage writes and publishing
Idle platform (7 services)	–	about 1.0 GB	Database 23 MB with AOI1 loaded

6.10 Iteration Outcome

Iteration 1 met its objectives. The architecture track delivered a working platform that reduces the data for one screen from 43.3 MB to 337 KB, serves cold tiles in 47 ms and cached tiles in 1 ms, analyses uploaded imagery end to end in under a minute, and accepts new models by folder and manifest. The model track established a fair evaluation protocol, showed that a time series is the key input and that model family matters little, selected a small CPU-only LightGBM pipeline (76.1% AOI1, 92.2% AOI2) and located the remaining errors on field edges. Measuring every workflow stage answered the panel's feasibility question: compute is small, memory is the limit, and the selected LightGBM model removes the 3.1 GB bottleneck of the baseline. The main open items are integrating the selected pipeline into the platform, the CDL confidence rule, the cross-area test, and the boundary and yield models.

Chapter 7

Testing and Early Validation

7.1 Testing Strategy

Testing follows the two tracks. The platform is tested from the smallest unit (the model registry) up to the whole system in a browser, with a dedicated performance benchmark because performance is the main design goal. The models are validated by an evaluation protocol designed to avoid optimistic results: spatial block split, validation-only decisions, a single test pass, intervals and per-class scores. Every defect found was fixed and re-tested, and the fix is recorded in the repository history.

Revised evaluation criteria. The panel found that the proposal's evaluation criteria did not measure the project's real challenges and named usability, performance and integration as the main concerns, together with a lightweight system (P2, P3). The criteria were therefore rewritten so that each concern has a metric, a target and a measured value (Table 7.1). Model accuracy remains a criterion, but is no longer the only one.

7.2 Test Levels and Techniques

7.3 Test Cases

7.4 Test Results

7.5 Acceptance Testing / Validation

The acceptance checks of Table 4.5 were run on the integrated system: all platform checks passed. Crop-type validation followed the protocol of Section 6.7.2 and is summarised in Table 6.8; live inference in the platform was also compared with the training-time raster for AOI1, and class areas agreed closely (Apples 6,927 ha in both; Alfalfa 11,340 ha against 10,781 ha, the difference coming from masking by the predicted rather than the reference

Table 7.1: Revised evaluation criteria and status at mid-term

Concern	Metric	Target	Measured (status)
Performance	Map data for one screen	<1 MB	337 KB (met)
	Cold / cached tile latency (median)	<300 / <50 ms	47 / 1 ms (met)
	Upload to map for an AOI-sized image	<2 min	57 s (met)
	Layers usable at once without lag	≥8	Not measured yet
Lightweight	Peak RAM per workflow stage	Fits a laptop	0.36–3.1 GB, fits 15 GB (met; Table 6.12)
	Selected crop-type model: size, CPU cost	CPU only, tens of MB	1.98 MB, 35.8 ms/1000 px (met)
	Whole platform on one host	One command	7 services, 1.0 GB idle (met)
Integration	Upload reaches the map without manual steps	All steps automatic	TC-11 (met)
	New model added by folder and manifest only	No other change	TC-03 (met)
	Layers from different models and images on one map	Same catalogue	6 layers on AOI1 (met)
Usability	Legends, readable errors, phone and desktop widths	NFR-U1–U3	Checked (met)
	Task success and questionnaire score (SUS) in a user study	≥90%, SUS ≥68	Planned, Iteration 2
Model quality	Accuracy and macro-F1 on spatial test blocks, with 95% intervals	Reported honestly	AOI1 76.1% / 0.686; AOI2 92.2% / 0.803

Table 7.2: Test levels and techniques

Level	What and how	Tooling
Unit	Registry discovery, dependency planning, error messages, placeholder behaviour, add-a-model flow, on small synthetic rasters.	pytest
Integration	Every API route through Nginx with real data in PostGIS and MinIO; tile responses and cache headers; PMTiles range reads with CORS.	curl, scripted checks
System / end-to-end	Upload to map; re-analysis; Live Map filter, field search and inspection; dashboard; cold start of the stack.	Headless and interactive browser
Performance	Data per screen, cold and cached tile latency, tile sizes, parse cost of the old approach.	`tools/benchmark_tiles.py`
Model validation	Spatial block test, bootstrap intervals, per-class scores, cost per 1000 pixels.	Evaluation notebooks

Table 7.3: Test cases (details in Appendix C)

ID	Test	Requirement
TC-01	Built-in models load without errors	FR-REG-01
TC-02	Dependencies resolve in order (crop type after cropland; yield after both)	FR-REG-03
TC-03	Add a model by copying the template and editing only the manifest	FR-REG-01, UC-11
TC-04	Placeholder model produces no data	FR-REG-04
TC-05	Raster tiles return PNG with immutable cache headers	FR-VIS-03
TC-06	Outside-raster tile returns 204; cached tile served by Nginx	FR-VIS-03, NFR-P2
TC-07	Statistics return area per class for the crop-type layer	FR-CAT-02
TC-08	Point query returns crop and cropland labels	FR-VIS-04
TC-09	Crop filter shows only the selected crop	FR-VIS-05
TC-10	PMTiles range read (206) with CORS	FR-CAT-03
TC-11	Upload Sentinel-2 image and run cropland + crop type	FR-ING-01..05
TC-12	Re-run NDVI model on an existing image	FR-ING-07
TC-13	Upload a 1-band file	FR-ING-03
TC-14	Rendering benchmark	NFR-P1, NFR-P2
TC-15	Cold start of the full stack	NFR-R1

Table 7.4: Test results

ID	Observed result	Status
TC-01–04	5 of 5 pytest cases passed in 0.4 s.	Pass
TC-05	Four raster layers returned 256×256 PNG tiles with `public, max-age=31536000, immutable`.	Pass
TC-06	Out-of-range tiles returned 204; repeated requests showed `X-Cache-Status: HIT`.	Pass
TC-07	Crop-type layer: 14 classes, 59,537 ha (147,118 acres) classified; Alfalfa 18.1%, Corn 12.1%, Apples 11.6%.	Pass
TC-08	Clicking a pivot field returned "Alfalfa" and "Cropland" (API) and "Spring Wheat" in another field (browser).	Pass
TC-09	Filter "Apples" left only apple pixels over the imagery.	Pass
TC-10	PMTiles range request returned 206 with `Access-Control-Allow-Origin`.	Pass
TC-11	Job completed in 57 s; all six steps done; both layers listed on the map.	Pass
TC-12	NDVI run completed in about 1 s; new layer listed.	Pass
TC-13	Job failed with "The image has 1 bands. Expected a Sentinel-2 stack (B2, B3, B4, B8, B11, B12 and optionally NDVI)."	Pass
TC-14	337 KB per screen, cold 47 ms median, cached 1 ms (Table 6.11).	Pass
TC-15	After `down` and `up -d`, all seven services healthy; health status "ok".	Pass

cropland). A supervised acceptance session with the supervisor and external co-supervisor is planned at the mid-term demonstration.

7.6 Edge and Failure Cases

- Non-GeoTIFF files are rejected before upload; images without a CRS or with an unsupported band count fail the job with a readable reason (TC-13).
- Placeholder models are skipped and marked "not available" instead of failing the job.
- Requests for tiles outside a raster return 204, which the map treats as empty.
- Points outside the data return "No data".
- A broken model manifest is reported on the Models and Status pages while other models load.
- Float imagery with NaN no-data values is handled (stored as null in the catalogue).

Table 7.5: Defects found during Iteration 1 and corrective actions

No.	Defect	Corrective action
1	Official MinIO images no longer available from public registries.	Pinned Bitnami build of the same MinIO server; documented.
2	Table name `layer` clashed with the PostGIS `topology.layer` table.	Renamed to `map_layer`; migrations limited to GeoCrop tables.
3	rasterio failed in slim containers (missing `libexpat`).	Added the system library to both images.
4	MapLibre web worker failed to load (wrong path, then wrong MIME type for `.mjs`).	Explicit worker URL; Nginx MIME type rule.
5	Bucket initialisation raced MinIO start-up on cold boot.	Retry loop in the initialiser; cold start retested.
6	Uploaded-image layers returned 404 tiles: NaN no-data could not be serialised.	Store non-finite no-data as null; repaired existing rows.
7	Browser kept an old page after a new build.	`no-cache` for the page shell, long cache for hashed assets.
8	Site lost the API after a backend container was rebuilt (stale address).	Nginx resolves services through Docker DNS at request time.

7.7 Defects and Corrective Actions

7.8 Current Validation Status

- **Platform:** all 15 test cases pass; performance targets NFR-P1, NFR-P2 and NFR-P4 are met on AOI1; NFR-P3 (eight or more layers visible at once) has not been measured yet.
- **Models:** crop-type pipeline validated on two areas with spatial blocks and intervals; not yet validated across areas or against field data; CDL confidence rule pending.
- **Not yet validated:** boundary accuracy (IoU, area error), yield metrics, multi-user load, and the integrated LightGBM pipeline's serving cost.

Chapter 8

Project Management and Development Evidence

8.1 Work Completed

- Literature review extended from 11 proposal papers to 15 summarised studies, with eight crop-type papers read for the model comparison.
- Proposal-stage prototype (Django, Leaflet, offline Random Forest scripts) analysed and its rendering bottleneck measured.
- New platform repository with seven containerised services, database migration, model registry, processing pipeline, tile service and Web GIS portal (Chapter 6).
- AOI1 data prepared as COG (imagery and CDL), baseline models retrained and saved for live inference, field-region PMTiles built, demonstration data loaded.
- Monthly 2022 Sentinel-2 composites for AOI1 and AOI2; spatial evaluation protocol; 11-model comparison with cost; improvement ladder; error analysis; comparison with papers.
- Rendering benchmark, 15 platform test cases, and project documentation (architecture, status, upgrade path, adding a model, running without Docker, demonstration script).

8.2 Work Remaining

- Integrate the LightGBM crop-type pipeline (monthly composites, neighbourhood inputs, vote) as a registry model; monthly-composite ingestion.
- CDL confidence rule; AOI2 interval and interior table; cross-area test; exports for the remaining AOIs.

- Field-boundary model and boundary/area validation (IoU, area error, acreage against NASS).
- Yield data preparation and yield model (R^2, RMSE, MAE).
- Multi-image upload, mosaics and an all-areas map view.
- Hardening (signed tile access, required sign-in, monitoring), deployment and final evaluation.

8.3 Individual Responsibilities

Table 8.1: Individual responsibilities

Member	Role	Responsibilities
Huzaifa Bin Khalid (22I-2332)	Architecture and platform	System architecture; performance analysis; data and storage design; tile delivery; processing pipeline; model registry; Web GIS portal; deployment configuration; benchmark and platform testing.
Saneed Ullah (23I-2568)	Crop-type model evaluation	Monthly composite preparation; evaluation protocol; model comparison and cost measurement; improvement experiments; error analysis; comparison with reference papers.
Taha Bashir Awan (23I-0824)	Crop-type model evaluation	Model experiments and comparison across families; evaluation and analysis of results; comparison with reference papers.

8.4 Contribution of Each Team Member

Huzaifa Bin Khalid defined the base architecture with performance as the main constraint, identified and measured the prototype's rendering problem, chose and integrated the technologies (COG, TiTiler, PMTiles, MapLibre, FastAPI, PostGIS, Celery, MinIO), designed the plug-in model registry and processing pipeline, delivered the Web GIS portal styled after the original prototype, and validated the system with the benchmark and end-to-end tests.

Saneed Ullah and Taha Bashir Awan carried out the crop-type model evaluation and comparison: the two study areas and their monthly composites, the spatial block protocol, the eleven-model comparison with speed, size and memory, the run-3 improvements (neighbourhood inputs and neighbour vote), the per-class and edge-pixel error analysis, and the comparison with eight reference papers, leading to the LightGBM recommendation. The crop-type results report was prepared by Saneed Ullah.

All members share integration, documentation and the final evaluation.

8.5 Repository and Version Control Evidence

- **Prototype repository** (`zzzzzzzzzzzzz`, Git): Django prototype, offline training scripts and proposal-stage results; 11 commits from 1 to 8 September 2026.
- **Platform repository** (`geocrop-platform`, Git, branch `main`): the Iteration 1 architecture and implementation, data tools, extended training scripts, tests and documentation; six commits on 14 October 2026 (Appendix D). Large data and model files are excluded from Git and reproduced with `makedata`.
- **Research notebooks:** `GeoCrop_CropType_Model_Benchmark.ipynb` (runs 1 and 2, model families, cost, bootstrap) and `GeoCrop_Accuracy_Boost.ipynb` (run 3), with the crop-type results report dated 14 October 2026.

8.6 Milestones and Commit Evidence

Table 8.2: Milestones

Date (2026)	Milestone	Evidence
1–8 Sep	Prototype: Django portal, Live Map, Analysis Studio, two-stage Random Forest	Prototype commits `f10912e–3760eae`
7–8 Sep	Proposal document	Proposal LaTeX source
11 Sep	Proposal defence	Proposal slides
Sep–Oct	Monthly composites; runs 1–3; paper comparison	Notebooks; results report (14 Oct)
14 Oct	Platform repository, services, registry, portal	Commits `f056a54–fae4751`
14 Oct	Reliability fixes; optional sign-in; UI after the prototype	Commits `efc3c68–4c631c9`
15 Oct	Mid-report	This document

8.7 Tools and Tool-Assisted Development

Standard tools were used throughout: Git for version control, Docker for reproducible services, Jupyter notebooks for experiments, Google Earth Engine for imagery export and LaTeX for documentation.

AI-assisted development. An AI coding assistant (Anthropic Claude, used through Claude Code) was used on the architecture track to speed up implementation: generating parts of the platform code and configuration from the team's architecture and requirements, diagnosing deployment problems, and drafting documentation, including parts of this report. The architecture, technology choices, requirements, acceptance criteria and evaluation targets were set by the team; all generated code was run, tested and reviewed against the tests and benchmark reported here, and the team remains responsible for it. This goes beyond the boundary

stated in the proposal (generative AI limited to documentation and presentation support) and is recorded as a deviation in Section 3.8. The models, preprocessing and evaluation design of the research track are the team's own work.

8.8 External Components and Dependencies

Table 8.3: External components, data and licences

Component	Use	Licence / terms
Sentinel-2 imagery	Model input	Copernicus open licence
USDA CDL, USDA NASS	Labels, yield statistics	U.S. public domain
OpenStreetMap tiles	Default basemap	ODbL, attribution shown
MapLibre GL JS, react-map-gl	Map rendering	BSD-3, MIT
React, Vite, TanStack Query, Zustand	Frontend	MIT
FastAPI, Pydantic, SQLAlchemy, Alembic	API	MIT
TiTiler, rio-tiler, rasterio, GDAL	Raster processing and tiles	MIT, BSD-3
PostgreSQL, PostGIS	Database	PostgreSQL licence, GPL-2
Celery, Redis	Jobs	BSD-3, BSD-3 (Redis 7)
MinIO (Bitnami build)	Object storage	AGPL-3
scikit-learn, LightGBM, PyTorch	Models	BSD-3, MIT, BSD-3
Presto (research comparison)	Pre-trained model	MIT
Nginx, Docker	Edge, containers	BSD-2, Apache-2

8.9 Risks and Mitigation

8.10 Security and Privacy Considerations

- Passwords are hashed with bcrypt; sessions use signed JWTs; sign-in can be made mandatory with one setting (currently optional so the portal opens directly, which is shown on the status page).

Table 8.4: Risks and mitigation

Risk	Level	Mitigation
CDL label noise and mixed edge pixels	High	Report all-pixel scores first; add one fixed CDL confidence rule; neighbourhood inputs and vote; field-level classification later.
Poor generalisation to new areas or years	High	Cross-area test in Iteration 2; more AOIs; model cards state valid regions.
Cloudy or missing imagery	Medium	Monthly composites with linear gap filling; validation of inputs at ingest.
Limited yield ground truth	High	Use county-level NASS data and evaluate at that scale; state granularity clearly.
Model size and serving cost	Medium	Baseline crop-type model measured at 3.1 GB RAM; replace with LightGBM (549 MB); measure every stage again after integration.
Usability not yet tested with users	Medium	Task-based user study with questionnaire in Iteration 2; findings fed into the portal.
Integration of research pipeline	Medium	Common plug-in contract; integration first in Iteration 2.
Dependency or registry changes (e.g. MinIO images)	Low	Pinned image versions; S3-compatible abstraction allows swapping.
Team time and coordination	Medium	Two parallel tracks with a shared contract; iteration reviews.

- Secrets are read from environment variables; the repository contains only an example file.
- Areas and layers are filtered by owner or public flag in every catalogue query.
- Upload URLs are signed, short-lived and limited to one object; only vector tiles are publicly readable in storage.
- The tile service applies styles from the database, not from request parameters; internal routes require a service token.
- Uploaded or partner-provided data is not published; result layers of uploads are private to the uploading account.

8.11 Current Limitations

- The platform runs the proposal-stage baseline crop-type model; the selected LightGBM pipeline is not integrated yet.
- Accuracy is agreement with the CDL for one year and two areas; no cross-area or field-survey validation yet.
- Field regions are derived from crop-type output, not from a boundary model.
- Yield estimation is not implemented.
- Upload supports one stacked GeoTIFF at a time; `.SAFE` products, cloud masking and mosaics are not supported yet.
- Tile URLs are not individually authorised; the default basemap is OpenStreetMap streets until a provider key is configured.
- The system runs on a single host; load with many concurrent users has not been tested.

Chapter 9

Remaining FYP Plan

9.1 Remaining Scope

The remaining scope completes the satellite-to-GIS chain on top of the platform built in Iteration 1: the selected crop-type pipeline in production use, a dedicated field-boundary model with area validation, yield estimation, wider evaluation across areas, and a hardened, deployed system.

9.2 Remaining Modules and Features

Table 9.1: Remaining modules and features

Module	Feature	Iteration
M1 Data preparation	Monthly-composite ingestion in the pipeline; exports for remaining AOIs; optional cloud mask	2, 3
M2 Crop mapping	LightGBM crop-type pipeline as a registry model; CDL confidence rule; cross-area test; field-level classification	2, 3
M3 Boundaries and area	Field-boundary model; polygon simplification; IoU and area error; acreage against NASS	3
M4 Yield	NASS yield preparation; yield model; county and field summaries	3, 4
M5 Processing pipeline	Multi-image upload; batch progress; mosaics	2
M8 Web GIS portal	All-areas view; compare runs; yield layers and dashboard; exports (GeoJSON, CSV)	2–4
Hardening	Required sign-in, signed tile access, monitoring, load test, provider basemap	4

9.3 Remaining Iterations

- **Iteration 2 (mid-Oct – Dec 2026, FYP-1 end):** crop-type pipeline integration, CDL confidence rule, AOI2 interval, cross-area test, multi-image upload and all-areas view, serving-cost measurement.
- **Iteration 3 (Feb – Mar 2027, FYP-2):** field-boundary model and area validation; yield data and first yield model; more AOIs; field-level crop classification experiment.
- **Iteration 4 (Apr – May 2027, FYP-2):** yield layers and dashboard, end-to-end evaluation against the success criteria, hardening, deployment, final report and demonstration.

9.4 Implementation Plan

New models are added through the registry (task and manifest), so the API, schema and interface remain stable. The LightGBM pipeline is packaged with its saved model, feature definition and metrics file; the ingest step gains a monthly-composite path. The boundary model writes a raster or polygons that the existing vector-tile builder publishes as PMTiles. The yield model reads the crop-type layer and satellite features and writes a continuous raster plus county summaries. The portal adds layer groups and dashboard panels driven by the catalogue.

9.5 Testing and Validation Plan

- Unit tests for each new model task (validation, outputs, metrics) and the extended pipeline.
- Integration and end-to-end tests repeated for every release; benchmark re-run with the new model and with several areas and layers visible.
- Crop type: all-pixel and CDL-confidence scores, cross-area test, per-class scores, intervals.
- Boundaries: IoU/Dice against reference boundaries and area error; acreage against USDA NASS.
- Yield: R^2, RMSE and MAE on held-out years or counties.
- Usability study (P3): five to eight participants, including the supervisors and, if possible, an agricultural user, complete set tasks (find a crop, compare two layers, upload an image); task success, time and a System Usability Scale questionnaire are recorded.
- Resource measurement repeated for every new model (time, peak RAM, storage), as in Table 6.12.

9.6 Deployment Plan

The same Docker Compose stack will be deployed to a single cloud virtual machine for the final demonstration, with a provider basemap key, required sign-in, HTTPS through the edge server and persistent volumes. The architecture allows managed PostgreSQL, S3 storage, a CDN in front of the tile path and additional worker and tile replicas if more users or areas are added.

9.7 Tentative Timeline

Table 9.2: Tentative timeline

Period	Planned work
Oct 2026	Mid-term demonstration; start crop-type pipeline integration
Nov 2026	Pipeline integrated; CDL confidence rule; cross-area test; multi-image upload
Dec 2026	Serving-cost measurement; FYP-1 final report and demonstration
Feb 2027	Boundary model; remaining AOI exports; yield data preparation
Mar 2027	Boundary and area validation; first yield model
Apr 2027	Yield in portal; end-to-end evaluation; hardening; deployment
May 2027	Final evaluation, report and demonstration

9.8 Expected Final Deliverables

- Deployed GeoCrop platform with crop-type, field-boundary and yield layers, dashboard and model catalogue.
- Source code repositories with tests, documentation and reproducible data preparation.
- Trained models with model cards and stored metrics.
- Evaluation report covering crop type, boundaries, area and yield, with comparison to published work.
- Final FYP report, presentation and demonstration.

Chapter 10

Conclusion

GeoCrop addresses the slow, manual and hard-to-scale process of mapping crops by field survey and GIS digitisation. Research showed that existing tools solve classification, mapping or yield in isolation, that many studies report optimistic accuracy from random splits, and that research prototypes become slow when results are shipped as large vector files: our own prototype needed 43.3 MB and 101,435 polygons for one area.

Requirements were defined for a processing pipeline, a plug-in model registry, a spatial catalogue and a Web GIS portal, with measurable performance targets. The design keeps model results as rasters, serves only visible tiles through TiTiler and PMTiles, runs heavy work in background workers and keeps a PostGIS catalogue that drives the interface. Iteration 1 implemented this architecture as seven containerised services. One screen of AOI1 now needs 337 KB, cold tiles return in 47 ms and cached tiles in 1 ms, an uploaded Sentinel-2 image is analysed and published in 57 s, and new models plug in without code changes elsewhere. On the model track, a fair spatial evaluation of eleven crop-type models showed that a monthly time series is the key input and that a small CPU-only LightGBM pipeline with neighbourhood inputs and a neighbour vote reaches 76.1% (AOI1) and 92.2% (AOI2), with most remaining errors on field edges.

All platform test cases pass and the main performance targets are met. The main risks are label noise in the CDL, generalisation to new areas, and limited yield ground truth. FYP-2 continues by integrating the selected crop-type pipeline, adding field-boundary and yield models, evaluating across areas, and deploying the hardened system.

Appendix A

Proposal Action Register

Table A.1: Complete proposal action register

No.	Source	Feedback / required change	Action taken	Status	Evidence
P1	Proposal panel	Bring clarity to intended goals.	Goals restated: light, fast, integrated platform plus fair, cheap crop-type models; crop mapping, GIS integration and "real-time" defined; objectives have measurable targets.	Done	Section 1.5
P2	Proposal panel	Improve evaluation metrics.	Evaluation criteria rewritten per concern with metric, target and measured value; spatial block protocol with intervals for models.	Done	Table 7.1, Sec. 6.7.2
P3	Proposal panel	Measure the real challenges: computational performance, lightweight system, usability; usability, performance and integration are the main concerns.	Performance benchmark; CPU cost, size and RAM for 11 models; integration tests; usability checked against NFR-U1–U3; user study planned.	Partly done	Tables 6.11, 6.6, 7.1

No.	Source	Feedback / required change	Action taken	Status	Evidence
P4	Proposal panel	Analyse complexity and resource requirements of the complete workflow beforehand.	Each stage measured for time, peak RAM and storage; baseline crop-type model (1.57 GB, 3.1 GB RAM) identified as the bottleneck; LightGBM (1.98 MB, 549 MB RAM) selected.	Done	Table 6.12
P5	Proposal panel	Running the entire process in a single flow may not be practical.	Separate queued stages in background workers, one model at a time in dependency order; per-step progress and errors; workers scale out.	Done	Sec. 6.9.1, Fig. 5.3
P6	Proposal panel	Huge datasets are not easy to load and use together.	Results kept as COGs; only visible tiles sent (337 KB per screen against 43.3 MB); boundaries as PMTiles; one layer catalogue.	Done	Table 6.11, Fig. 6.4
P7	Proposal panel	Work on presentation skills.	Fixed demonstration script on the running system; each member presents their own track; rehearsals before each evaluation.	In progress	Sec. 6.9
T1	Team review	Uploads were recorded but never analysed.	Processing pipeline with validation, COG conversion and background model runs; 57 s upload to map.	Done	TC-11, Fig. 5.3
T2	Team review	Dashboard used hand-written sample data.	Statistics computed from real result rasters and stored in PostGIS.	Done	TC-07, Fig. 6.7
T3	Team review	First evaluation used a random pixel split.	Spatial block split with buffer, validation-only choices, block bootstrap intervals.	Done	Sec. 6.7.2
T4	Team review	Models will change over the project.	Plug-in registry with manifests and dependency graph; add-a-model test.	Done	TC-03

No.	Source	Feedback / required change	Action taken	Status	Evidence
T5	Team review	Slide iteration plan outdated.	Two-track iteration plan with four iterations.	Done	Table 3.2
T6	Team review	Use the original prototype's look in the new portal.	Portal restyled after the prototype (home page, workspace, panels).	Done	Figs. 6.5, 6.6
T7	Team review	Integrate the selected crop-type pipeline into the platform.	Planned as first item of Iteration 2.	Open	Section 3.5
T8	Team review	Like-for-like comparison with Tran et al.	CDL confidence rule and class alignment in Iteration 2.	Open	Section 6.7.6
T9	Team review	Usability study with users.	Task-based study with SUS questionnaire.	Open	Section 3.5

Appendix B

Detailed Requirements

Table B.1: Remaining use cases in brief

ID	Actor	Main flow	Alternative flows
UC-03	Agricultural Analyst	Choose a crop in the Filter chip or click a crop in the ranking; map shows only that crop; type a crop name to search field regions and click one to fly to it.	No match: “No field regions match”.
UC-04	Agricultural Analyst	Open Dashboard; choose area, layer and unit; read classified area, class shares, breakdown table and model accuracy.	Area without categorical layers: empty state.
UC-06	GIS Analyst	After an upload or run, the Processing page polls the job and shows steps, progress and messages; on completion links to the Live Map.	Failed job: reason shown; link to analyse another image.
UC-07	GIS Analyst	Choose an image already in GeoCrop and models; a run job starts; layers are added.	Unknown model or missing dependency: readable error.
UC-08	Any user	Open Models; read status, inputs, outputs, valid regions, accuracy and F1 per class.	Registry errors listed.
UC-09	Any user	Create an account (all fields, matching passwords, unique username and email) or log in.	Duplicate username or email; wrong password.
UC-10	Platform Developer	Open System Status; see health of database, queue, storage, tiles and registry, plus configuration notes.	Failed service shown in red with detail.
UC-12	Processing Worker	Take a job from the queue; run ingest or model steps; publish layers and statistics; update progress.	Retry on transient errors; mark failed with reason.

Table B.2: Requirement priority and status

Requirement	Priority	Status at mid-term
FR-ING-01..07	Must	Implemented and tested (FR-ING-06 stores location as area name and capture date on the image)
FR-REG-01..05	Must	Implemented and tested
FR-MAP-01..02	Must	Baseline implemented; selected pipeline in Iteration 2
FR-CAT-01..04	Must	Implemented; FR-CAT-03 uses derived field regions
FR-VIS-01..08	Must	Implemented and tested
FR-AUT-01	Should	Implemented; optional by configuration
NFR-P1, P2, P4	Must	Met (Table 4.5)
NFR-P3	Should	Not yet measured
NFR-P5	Must	Met by the selected model (2 MB, CPU); integration pending
NFR-R1..R4, U1..U4, S1..S5	Must	Implemented; S3 tile-level authorisation partial

Appendix C

Detailed Test Cases

Table C.1: Detailed test cases

ID	Preconditions / input	Steps	Expected result	Result
TC-01	Built-in model folders	Load the registry	No errors; cropland, croptype, boundary, yield, NDVI found	Pass
TC-02	Registry	Plan `croptype`; plan `yield`	[cropland, croptype]; [cropland, croptype, yield]; unknown and cyclic plans give readable errors	Pass
TC-03	Copy of the NDVI template with a new key	Load both folders; plan and run on a 64×64 synthetic 7-band raster	Model discovered; one float32 output, values ≤ 1	Pass
TC-04	Yield placeholder	Run on synthetic raster	Status "not available"; no files	Pass
TC-05	Seeded AOI1 layers	Request a z12 tile of each raster layer	200, PNG 256×256, immutable cache header	Pass
TC-06	Same	Request a tile outside the data; repeat a tile	204; second request `X-Cache-Status:HIT`	Pass
TC-07	Crop-type layer	`GET/stats?layer=`	14 classes; areas sum to total; shares	Pass
TC-08	Crop-type and cropland visible	`GET/point` at a pivot field; click in browser	Labels returned (Alfalfa / Cropland; Spring Wheat)	Pass

ID	Preconditions / input	Steps	Expected result	Result
TC-09	Live Map	Choose “Apples” in Filter	Only apple pixels remain	Pass
TC-10	Field regions layer	Range request with Origin header to the PMTiles URL	206 Partial Content; CORS header	Pass
TC-11	AOI1 Sentinel-2 GeoTIFF (7 bands)	Signed PUT; complete with cropland and crop type; poll job	Done in under 2 min; 6 steps done; layers listed	Pass (57 s)
TC-12	Existing image	Re-run NDVI from Analyze Imagery	Processing page shows done; layer listed	Pass
TC-13	1-band CDL GeoTIFF	Upload and complete	Job failed with band-count reason	Pass
TC-14	Running stack	`makebenchmark`	Under 1 MB per screen; cold under 300 ms; cached under 50 ms	Pass
TC-15	Stopped stack	`dockercomposeup-d`; `GET/health`	All services healthy; status ok	Pass

Table C.2: AOI1 per-class results (improved pipeline with neighbour vote; test pixels)

Class	Test px	On edges	F1 bench.	F1 final	Recall int.	Recall edge
Alfalfa	19,789	40.0%	0.829	0.845	0.960	0.504
Corn	14,830	41.5%	0.821	0.819	0.965	0.637
Potatoes	14,277	35.1%	0.883	0.860	0.971	0.684
Other Hay	9,061	58.6%	0.707	0.722	0.949	0.535
Winter Wheat	7,703	47.3%	0.774	0.792	0.967	0.598
Apples	7,698	60.6%	0.620	0.644	0.866	0.415
Sweet Corn	7,149	44.1%	0.682	0.709	0.963	0.584
Dry Beans	5,345	50.8%	0.745	0.743	0.938	0.714
Other (rare)	3,598	82.6%	0.239	0.287	0.595	0.110
Peas	2,751	72.0%	0.509	0.538	0.858	0.589
Sod/Grass Seed	2,490	42.2%	0.715	0.706	0.951	0.731
Onions	2,292	25.9%	0.911	0.913	0.998	0.737
Spring Wheat	1,506	51.3%	0.550	0.741	0.997	0.429
Fallow/Idle	749	99.5%	0.269	0.289	n/a	0.517

Table C.3: AOI2 per-class results (improved pipeline with neighbour vote)

Class	Test px	Recall	Precision	F1 final	F1 bench.
Canola	52,527	0.979	0.975	0.977	0.972
Spring Wheat	47,912	0.921	0.955	0.938	0.925
Soybeans	20,268	0.912	0.902	0.907	0.901
Fallow/Idle	14,051	0.840	0.841	0.840	0.817
Barley	3,255	0.614	0.740	0.671	0.581
Dry Beans	2,584	0.886	0.825	0.854	0.792
Peas	2,255	0.875	0.620	0.726	0.707
Other (rare)	1,259	0.659	0.414	0.509	0.428
Sunflower	811	0.783	0.837	0.809	0.712

Table C.4: Fair reporting of AOI1 accuracy by subset of test pixels (accuracy % / macro-F1)

Pixels counted	Share	Benchmark	Improved	+ vote
All test pixels (headline)	100.0%	74.1 / 0.661	75.6 / 0.685	76.1 / 0.686
Same-label patch $\geq$ 10 px (0.9 ha)	84.5%	83.3 / 0.725	85.0 / 0.756	86.4 / 0.771
Same-label patch $\geq$ 50 px (4.5 ha)	79.2%	86.6 / 0.807	88.3 / 0.849	90.0 / 0.867
Interior pixels (3$\times$3 same label)	53.0%	93.8 / 0.825	94.9 / 0.853	95.2 / 0.856
Deep interior (5$\times$5 same label)	36.2%	96.4 / 0.927	97.1 / 0.952	97.4 / 0.955

Subsets use only the reference labels, never model output; they describe where errors are and are not a better score. The all-pixel number is always quoted first.

Appendix D

Repository and Contribution Evidence

Table D.1: Platform repository history (`geocrop-platform`, branch `main`)

Commit	Date	Message
`f056a54`	2026-10-14	Step 0: repo skeleton, env example, data fetch script
`1ec3dc9`	2026-10-14	Backend, model registry, tile service, React app, compose stack and docs
`fae4751`	2026-10-14	Initial migration, Docker fixes, MapLibre worker, verified end to end with benchmark
`efc3c68`	2026-10-14	Retry MinIO connection in bucket init (fixes cold-start race)
`1e416ed`	2026-10-14	Open straight to the map without sign-in; fix NaN nodata and stale page cache
`4c631c9`	2026-10-14	Restyle the web app after the original GeoCrop prototype

Table D.2: Prototype repository history (proposal stage)

Commit	Date	Message
`f10912e`	2026-09-01	Initial Django project
`4dfe6b3`	2026-09-05	Update frontend to look slight professional
`02f9e00`	2026-09-05	Add render setup
`bab4d9e`	2026-09-05	Add Django dependencies
`0788e6f`	2026-09-05	Trigger Vercel deployment
`b4ed861`	2026-09-05	Add requirements.txt for Render deployment
`b9b63cd`	2026-09-08	Add changes
`3760eae`	2026-09-08	Add changes to the crop result file

Platform repository structure.

```
geocrop-platform/
  backend/    app/ (api, core, db, services, workers, cli)
              model_registry/, alembic/, tests/
```

```
tiles/     TiTiler-based tile service
web/       React app and nginx.conf
ml/        extended training scripts
tools/     to_cog.py, build_boundaries.py,
           benchmark_tiles.py
infra/     MinIO initialisation, Nginx reference
docs/      architecture, status, upgrade path, report
docker-compose.yml, Makefile, .env.example
```

Appendix E

Additional Design Diagrams

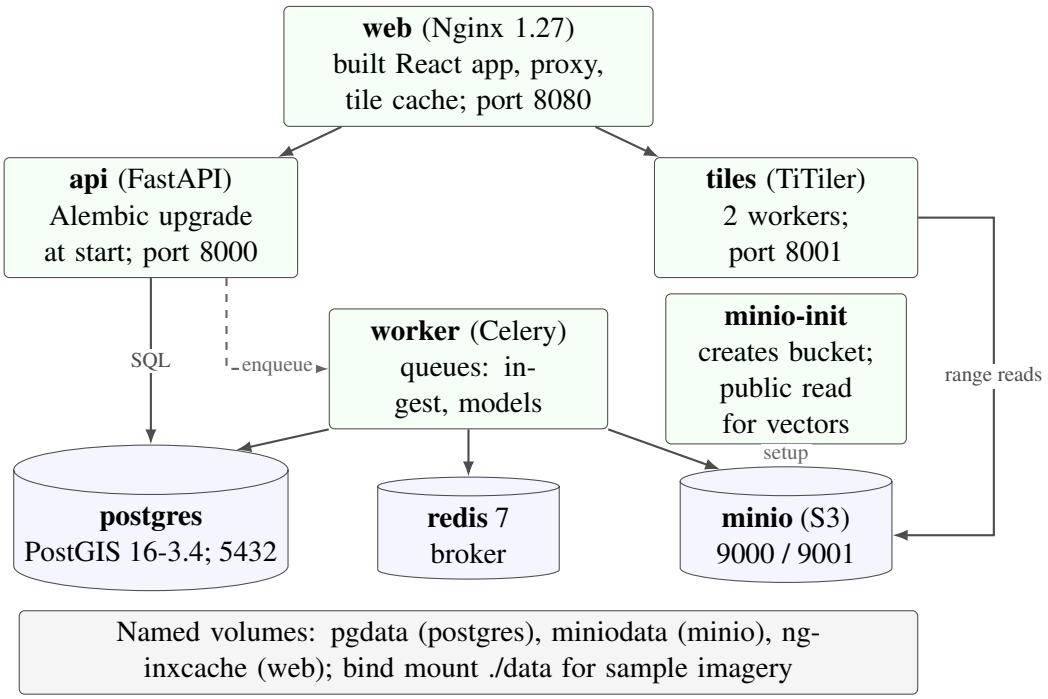


Figure E.1: Deployment view of the Docker Compose stack

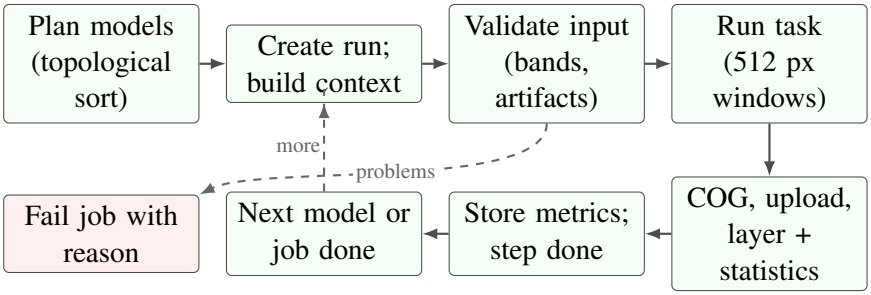


Figure E.2: Activity of a model run inside the worker

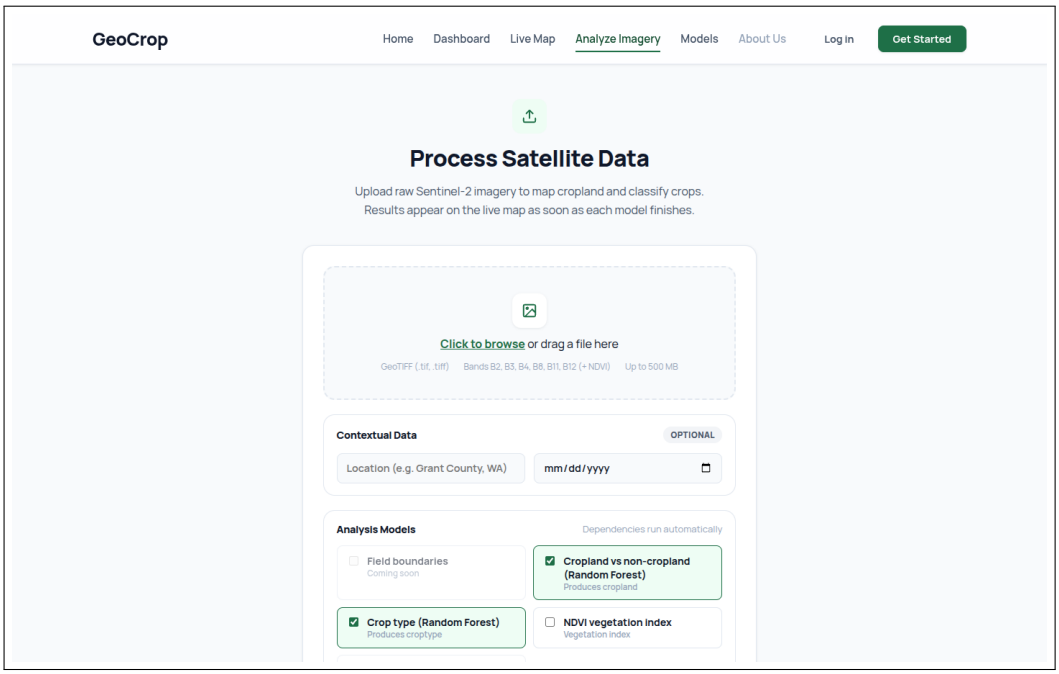


Figure E.3: Analyze Imagery: drag-and-drop upload, contextual data and model selection

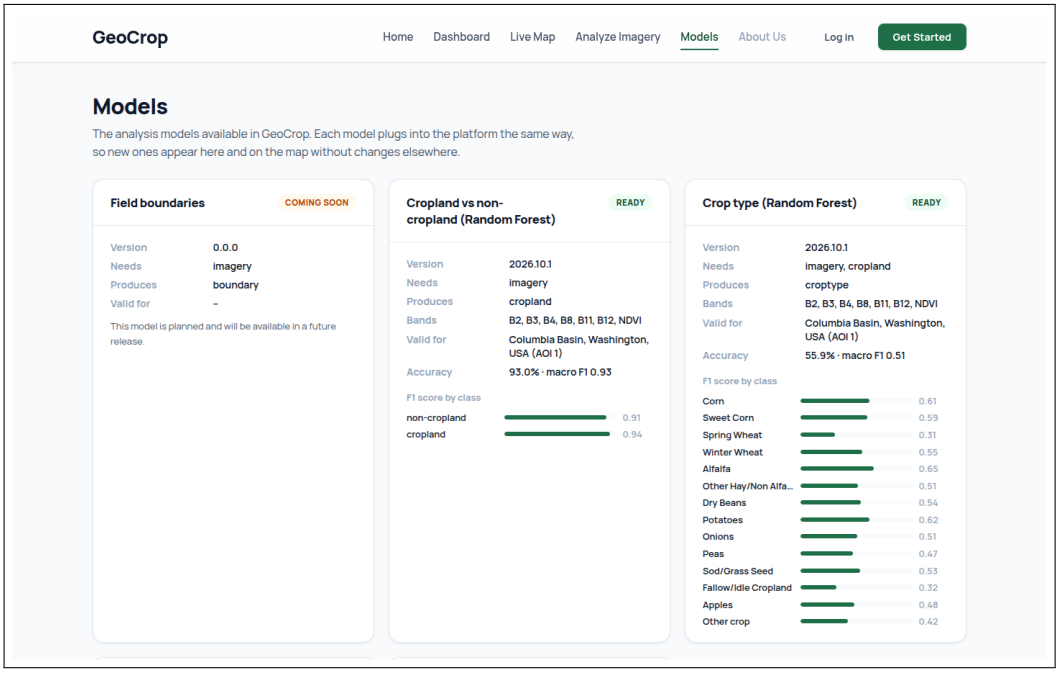


Figure E.4: Models page: registered models, status, accuracy and F1 per class

Appendix F

Meeting and Progress Evidence

Table F.1: Progress evidence from dated project artefacts

Date (2026)	Progress	Artefact
1–8 Sep	Proposal-stage prototype implemented	Prototype repository commits
7–8 Sep	Proposal written	Proposal document and LaTeX source
11 Sep	Proposal defence	Proposal slides (F26-155-GeoCrop)
14 Sep	Defence result: Approved with Minor Modifications; panel comments P1–P7	FYP Committee e-mail
Sep–Oct	Crop-type experiments, runs 1–3	Research notebooks
14 Oct	Crop-type results and paper comparison	Results report (prepared by Saneed Ullah)
14 Oct	Platform architecture implemented and verified	Platform repository commits; benchmark
15 Oct	Workflow resource measurement (P4)	Measurement script; Table 6.12
15 Oct	Mid-report	This document

Open questions prepared for the supervisors: (1) whether LightGBM with neighbourhood inputs and a 3×3 vote is accepted as GeoCrop's crop-type model; (2) whether the final comparison with the papers should use the CDL confidence filter, reported next to the all-pixel score; (3) whether field-level classification with field boundaries or the cross-area test should come first.

Table F.2: Supervisor meeting record (to be completed and signed by the team)

Date	Attendees	Discussion and decisions	Supervisor signature

Bibliography

[1] L. Lin, L. Di, C. Zhang, L. Guo, Y. Di, H. Li, and A. Yang, "Validation and refinement of cropland data layer using a spatial-temporal decision tree algorithm," *Scientific Data*, vol. 9, art. 63, 2022. doi: 10.1038/s41597-022-01169-w.

[2] P. Hao, L. Di, C. Zhang, and L. Guo, "Transfer learning for crop classification with Cropland Data Layer data (CDL) as training samples," *Science of the Total Environment*, vol. 733, art. 138869, 2020. doi: 10.1016/j.scitotenv.2020.138869.

[3] K. H. Tran, H. K. Zhang, J. T. McMaine, X. Zhang, and D. Luo, "10 m crop type mapping using Sentinel-2 reflectance and 30 m cropland data layer product," *International Journal of Applied Earth Observation and Geoinformation*, vol. 107, art. 102692, 2022. doi: 10.1016/j.jag.2022.102692.

[4] S. Wang, S. Di Tommaso, J. M. Deines, and D. B. Lobell, "Mapping twenty years of corn and soybean across the US Midwest using the Landsat archive," *Scientific Data*, vol. 7, art. 307, 2020. doi: 10.1038/s41597-020-00646-4.

[5] H. R. Khan, Z. Gillani, M. H. Jamal, A. Athar, M. T. Chaudhry, H. Chao, Y. He, and M. Chen, "Early identification of crop type for smallholder farming systems using deep learning on time-series Sentinel-2 imagery," *Sensors*, vol. 23, no. 4, art. 1779, 2023. doi: 10.3390/s23041779.

[6] R. M. A. Latif, J. He, and M. Umer, "Mapping cropland extent in Pakistan using machine learning algorithms on Google Earth Engine cloud computing framework," *ISPRS International Journal of Geo-Information*, vol. 12, no. 2, art. 81, 2023. doi: 10.3390/ijgi12020081.

[7] S. Muqaddas, W. S. Qureshi, H. Jabbar, A. Munir, and A. Haider, "A comprehensive deep learning approach for harvest ready sugarcane pixel classification in Punjab, Pakistan using Sentinel-2 multispectral imagery," *Remote Sensing Applications: Society and Environment*, vol. 35, art. 101225, 2024. doi: 10.1016/j.rsase.2024.101225.

[8] A. Edri, Y. Edan, L. Fine, O. Rozenstein, and T. Paz-Kagan, "Scalable field boundary refinement from satellite time series using deep learning," *Precision Agriculture*, vol. 27, art. 111, 2026. doi: 10.1007/s11119-026-10413-x.

[9] J. Desloires, D. Ienco, and A. Botrel, "Out-of-year corn yield prediction at field-scale using Sentinel-2 satellite imagery and machine learning methods," *Computers and Electronics in Agriculture*, vol. 209, art. 107807, 2023. doi: 10.1016/j.compag.2023.107807.

[10] G. Xiao, X. Zhang, Q. Niu, X. Li, X. Li, L. Zhong, and J. Huang, "Winter wheat yield estimation at the field scale using Sentinel-2 data and deep learning," *Computers and Electronics in Agriculture*, vol. 216, art. 108555, 2024. doi: 10.1016/j.compag.2023.108555.

[11] J. You, X. Li, M. Low, D. B. Lobell, and S. Ermon, "Deep Gaussian process for crop yield prediction based on remote sensing data," in *Proc. AAAI Conference on Artificial Intelligence*, 2017, pp. 4559–4565. doi: 10.1609/aaai.v31i1.11172.

[12] H. K. Zhang, Y. Shen, X. Zhang, J. Li, Z. Yang, Y. Xu, C. Zhang, L. Di, and D. P. Roy, "Robust and timely within-season conterminous United States crop type mapping using Landsat Sentinel-2 time series and the transformer architecture," *Remote Sensing of Environment*, vol. 329, art. 114950, 2025. doi: 10.1016/j.rse.2025.114950.

[13] H. Li, L. Di, C. Zhang, L. Lin, L. Guo, E. G. Yu, and Z. Yang, "Automated in-season crop-type data layer mapping without ground truth for the conterminous United States based on multisource satellite imagery," *IEEE Transactions on Geoscience and Remote Sensing*, vol. 62, art. 4403214, 2024. doi: 10.1109/TGRS.2024.3361895.

[14] D. Szwarcman *et al.*, "Prithvi-EO-2.0: A versatile multi-temporal foundation model for Earth observation applications," arXiv:2412.02732, 2024.

[15] V. Sainte Fare Garnot, L. Landrieu, S. Giordano, and N. Chehata, "Satellite image time series classification with pixel-set encoders and temporal self-attention," in *Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020. arXiv:1911.07757.

[16] G. Tseng, R. Cartuyvels, I. Zvonkov, M. Purohit, D. Rolnick, and H. Kerner, "Lightweight, pre-trained transformers for remote sensing timeseries," arXiv:2304.14065, 2023.

[17] M. Rußwurm, C. Pelletier, M. Zöllner, S. Lefèvre, and M. Körner, "BreizhCrops: A time series dataset for crop type mapping," *ISPRS Archives*, vol. XLIII-B2-2020, pp. 1545–1551, 2020. doi: 10.5194/isprs-archives-XLIII-B2-2020-1545-2020.

[18] C. Pelletier, G. I. Webb, and F. Petitjean, "Temporal convolutional neural network for the classification of satellite image time series," *Remote Sensing*, vol. 11, no. 5, art. 523, 2019. doi: 10.3390/rs11050523.

[19] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "LightGBM: A highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[20] C. Zhang, L. Di, L. Lin, and L. Guo, "Machine-learned prediction of annual crop planting in the U.S. Corn Belt based on historical crop planting maps," *Computers and Electronics in Agriculture*, vol. 166, art. 104989, 2019. doi: 10.1016/j.compag.2019.104989.

[21] Cloud Optimized GeoTIFF, "COG specification and documentation," 2026. [Online]. Available: https://cogeo.org.

[22] Development Seed, "TiTiler: a modern dynamic tile server built on FastAPI and Rasterio/GDAL," 2026. [Online]. Available: https://developmentseed.org/titiler/.

[23] Protomaps, "PMTiles: a single-file archive format for tiled data," 2026. [Online]. Available: https://docs.protomaps.com/pmtiles/.

[24] MapLibre, "MapLibre GL JS documentation," 2026. [Online]. Available: https://maplibre.org.

[25] PostGIS Project, "PostGIS: spatial and geographic objects for PostgreSQL," 2026. [Online]. Available: https://postgis.net.

[26] European Space Agency, "Sentinel-2 mission overview," 2026. [Online]. Available: https://sentiwiki.copernicus.eu/web/s2-mission.

[27] USDA National Agricultural Statistics Service, "Cropland Data Layer," 2026. [Online]. Available: https://www.nass.usda.gov/Research_and_Science/Cropland/SARS1a.php.

[28] Seoul Accord, "Section D – Graduate attributes: D.4 Common range and contextual definitions," 2026. [Online]. Available: https://www.seoulaccord.org/document.php?id=79.

[29] OneSoil, "OneSoil platform: field monitoring, productivity analysis and precision agriculture," 2026. [Online]. Available: https://onesoil.ai/en/platform.